ImgWindow Contrib Library

Modern UI Widgets & Extensions for Dear ImGui in X-Plane Plugins

---

ImGui Contributions in ImgWindow (`contrib/`)

Welcome to the **`contrib/`** directory of **ImgWindow**. This directory houses ImgWindow's bundled library of community-contributed **Dear ImGui (`ImGui::`) extensions and immediate-mode widgets** for X-Plane plugins.

■ Why `contrib/`? (From Debug Menus to Cockpit Instruments)

Pure Dear ImGui is fundamentally an engine designed for game-dev debug menus and developer panels. When flight sim plugin authors try to build user-facing products with it, they immediately hit friction:

- **Tooltips follow the mouse, flicker, and fall off the screen** when moving near the simulator window edges.
- **Help text in tooltips is limited to plain ASCII strings** without headers, bullet lists, bold emphasis, or documentation links.
- **Toggles are clunky checkboxes** rather than tactile, cockpit-grade switches with clear dual-state affordances.
- **Sliders and combos don't respond naturally to the mouse scroll-wheel**, making fine tuning feel clumsy compared to real avionics knobs.
- **There is no standard "revert to default" pattern**, so users fear tweaking settings lest they lose their calibrated values.

Look at the spectrum of UI tasks a flight sim plugin author faces, and how **`contrib/`** covers them today:

Common Cockpit UI Task	Vanilla Dear ImGui Friction	`contrib/` Solution
Interactive Help & Manuals	**`SetTooltip()`** follows the mouse cursor, flickers, and clips outside simulator window boundaries. Limited to plain ASCII strings.	**`ImGui::TimedTooltip`** & **`ImGui::MD`** Stationary anchor after initial delay, low-water wheel reset, smart cursor-avoidance and screen-edge clamping, and full Markdown rendering.
Two-Position Switches	Clunky **`[x]`** checkboxes with no visual state distinction or dual-label tooltips.	**`ImGui::ToggleButton`** Animated physical rocker/toggle switches with font-relative scaling, custom palettes, and automatic dual-state tooltips (**`"Off Hint", "On Hint"`**).
Avionics Dials & Adjusters	Requires click-and-drag. The mouse wheel does nothing, requiring awkward custom tracking.	**`ImGui::MouseWheelSlider`** / **`MouseWheelDrag`** / **`MouseWheelCombo`** Natural mouse-wheel stepping with configurable increments and fine-tune modifier keys.
Fearless Settings Tweaking	No standard "revert to default" pattern; users hesitate to adjust complex tuning parameters.	**`ImGui::AdjustItemOnMouseWheel`** & **Reset Chasers** Inline 1-click restore-to-default icon chaser with instant undo history.
External Docs & Web Links	Raw text URLs or awkward styled buttons with default arrow cursors.	**`ImGui::ButtonLink`**, **`TextLink`**, **`MenuItemLink`** Native OS hand cursors (**`ImGuiMouseCursor_Hand`**), hover underlines, and one-click browser launch via **`XPLMOpenURL`**.

■ Quick Start: Just `#include "contrib.h"`

Using **`contrib/`** requires zero setup or boilerplate. Simply include the master header:

```
[CPP]
#include "contrib.h"
```

All widgets and extensions immediately feel like native Dear ImGui:

```
[CPP]
// 1. Cockpit widgets live directly in ImGui:::
static bool s_HudActive = true;
ImGui::ToggleButton("HUD Active", &amp;s_HudActive);

static float s_Brightness = 0.85f;
ImGui::MouseWheelSlider("Brightness", &amp;s_Brightness, 0.0f, 1.0f, "%.0f%%");

// 2. Extensions mirror standard ImGui text &amp; tooltip vocabulary:
ImGui::Markdown::Text("# Flight Deck\nTelemetry is **active**.");
ImGui::TimedTooltip::Text("Hover-delayed stationary tooltip.");
```

TIP

Forgiving Aliases ("It Just Works") Don't sweat memorizing rigid namespace rules. The library includes transparent aliases so whatever you intuitively reach for compiles cleanly: * Prefer brevity? **`ImGui::MD::Text(...)`** is an official shorthand alias for **`ImGui::Markdown::Text(...)`**. * Prefer root access? **`ImGui::TextMD(...)`** and **`ImGui::TextMarkdown(...)`** work directly under **`ImGui::`**. * Combining tooltips and markdown? **`ImGui::TimedTooltip::MD::Text`** and **`ImGui::MD::TimedTooltip::Text`** work identically. * Want to skip **`ImGui::`** completely? **`#define CONTRIB_ENABLE_GLOBAL_NAMESPACES`** before including **`contrib.h`** to use **`Markdown::`**, **`MD::`**, and **`TimedTooltip::`** directly in the global scope.

Ecosystem Organization

To keep contributions modular and easy to navigate, the library is organized into two primary categories:

```
ImgWindow/
■■■ contrib.h                          # Master concentrator (-&gt; contrib/includes.h)
■■■ contrib/
    ■■■ includes.h                     # Master roll-up for all extensions &amp; widgets
    ■■■ contrib-global.h               # Opt-in global namespace aliases
    ■■■ extensions.h                   # Concentrator for all extensions (-&gt; extensions/includes.h)
    ■■■ widgets.h                      # Concentrator for all widgets    (-&gt; widgets/includes.h)
    ■
    ■■■ extensions/                    # Feature-complete ImGui::&lt;Package&gt;:: subsystems
    ■   ■■■ TimedTooltip/             # Stationary &amp; delayed tooltip engine with auto-bounds clamping
    ■   ■■■ Markdown/                 # Markdown parser &amp; renderer with dynamic font scaling &amp; tags
    ■
    ■■■ widgets/                       # Lightweight, immediate-mode ImGui:: UI controls
        ■■■ imgui_toggle_buttons.h     # Animated toggle switches &amp; segmented button pairs
        ■■■ imgui_mouse_wheel_controls.h # Sliders, drags, combos &amp; chasers with mouse-wheel support
        ■■■ imgui_clickable_links.h    # ButtonLink, TextLink, MenuItemLink &amp; AttachItemCursor affordances
        ■■■ imgui_badges.h             # Standalone C++ status badges, pills, and 3D keycaps (&lt;kbd&gt;)
        ■■■ imgui_spinners.h           # Procedural activity spinners &amp; radial countdown dials
```

1. Packages & Extensions (`contrib/extensions/`)

Feature-rich additions that manage internal state or background configuration (**`ImGui::Markdown::`**, **`ImGui::TimedTooltip::`**). See → contrib/extensions/README.md.

2. Reusable Widgets (`contrib/widgets/`)

Lightweight, immediate-mode UI controls that live directly in the root `ImGui::` namespace (`ImGui::ToggleButton`, `ImGui::MouseWheelSlider`, `ImGui::ButtonLink`). See → contrib/widgets/README.md.

■■ Configuration Reference: Opt-Out Flags

Everything in `contrib/` is **active and ready by default**. If your project happens to define a conflicting identifier, define any of the following macros *before* including `contrib.h` to selectively disable individual features:

Macro Directive	Type	Default	Description
`CONTRIB_ENABLE_GLOBAL_NAMESPACES`	**Opt-In**	*Off*	Publishes `Markdown::`, `MD::`, and `TimedTooltip::` to global namespace.
`CONTRIB_DISABLE_EXTENSIONS`	**Opt-Out**	*Off*	Excludes all extensions (`contrib/extensions/`).
`CONTRIB_DISABLE_WIDGETS`	**Opt-Out**	*Off*	Excludes all contributed widgets (`contrib/widgets/`).
`CONTRIB_DISABLE_TIMED_TOOLTIP`	**Opt-Out**	*Off*	Excludes the `TimedTooltip` extension.
`CONTRIB_DISABLE_MARKDOWN`	**Opt-Out**	*Off*	Excludes the `Markdown` extension.
`CONTRIB_DISABLE_TEXT_MD`	**Opt-Out**	*Off*	Excludes root `ImGui::TextMD` / `TextWrappedMD` aliases.
`CONTRIB_DISABLE_TOGGLE_BUTTONS`	**Opt-Out**	*Off*	Excludes `ToggleButton` and `ToggleButtons`.
`CONTRIB_DISABLE_TOGGLE_MOUSE_WHEEL`	**Opt-Out**	*Off*	Excludes mouse-wheel toggling on `ToggleButton`.
`CONTRIB_DISABLE_MOUSE_WHEEL_CONTROLS`	**Opt-Out**	*Off*	Excludes mouse-wheel sliders, drags, cyclers, and combos.
`CONTRIB_DISABLE_CLICKABLE_LINKS`	**Opt-Out**	*Off*	Excludes `ButtonLink`, `TextLink`, `MenuItemLink`, and `AttachItemCursor`.
`CONTRIB_DISABLE_BADGES`	**Opt-Out**	*Off*	Excludes standalone `Badge`, `Pill`, and `Kbd` widgets.
`CONTRIB_DISABLE_SPINNERS`	**Opt-Out**	*Off*	Excludes `Spinner`, `RadialTimer`, and `CountdownTimer`.
`CONTRIB_DISABLE_PERCENT_SLIDER`	**Opt-Out**	*Off*	Excludes `PercentSlider` and `SliderPercent`.
`CONTRIB_DISABLE_MOUSE_WHEEL_COMBO`	**Opt-Out**	*Off*	Excludes `MouseWheelCombo` and `ComboWithReset`.
`IMGUI_CONTRIB_DISABLE_RESET_UNDO`	**Opt-Out**	*Off*	Disables 1-click Reset-with-Undo on reset buttons.

(Note: Prefixes `IMGUI_CONTRIB_DISABLE_*`, `IMGUI_DISABLE_CONTRIB_*`, and legacy `IMGUI_DISABLE_EXTRA_*` are also honored automatically.)

How to Contribute

We welcome community contributions! Whether porting an existing open-source widget or introducing a new UI component:

1. **Choose the Category:** Determine if your code is a standalone **Widget** (a single control in `widgets/`) or a full **Extension** (a package in `extensions/`).
2. **Namespace Appropriately:** Follow the namespacing rules above.
3. **Keep Dependencies Minimal:** Header-only or clean single-file implementations are strongly preferred.
4. **Document Your Work:** Include a dedicated `README.md` in your extension folder detailing usage, API signatures, and code examples.
5. **Open a PR:** Submit a Pull Request against the repository with a description and sample screenshot or code snippet.

ImGui Extra Widgets

This directory contains lightweight, reusable, immediate-mode **Widgets** extending Dear ImGui within the **ImgWindow** framework.

Unlike **Extensions** (which manage their own complex background subsystems, registries, and configuration structures), **Widgets** are self-contained immediate-mode controls. They hook directly into Dear ImGui's ambient context (`GImGui`, ID stack, `ImGuiStorage`, and window draw lists) and live directly in the root `ImGui::` namespace to feel completely native.

Architecture & Inclusion Strategy

1. Root `ImGui::` Namespacing

All widgets in this directory are injected directly into `namespace ImGui { ... }`. You call `ImGui::ToggleButton(...)`, `ImGui::SliderWithReset(...)`, `ImGui::PercentSlider(...)`, and `ImGui::ButtonLink(...)` just like Dear ImGui's built-in `ImGui::Checkbox` or `ImGui::SliderFloat`.

2. The 4-Tier Inclusion Model

Pattern	Usage	Benefit
Master Contrib Roll-Up	`#include "ImgWindow/contrib.h"` *(or* `"contrib/includes.h"`*)*	Includes **all** extensions (`TimedTooltip`, `Markdown`) and contributed widgets in guaranteed dependency order.
Widgets Roll-Up	`#include "ImgWindow/contrib/widgets.h"` *(or* `"contrib/widgets/includes.h"`*)*	Instant access to the entire widget collection in one line.
A-la-Carte	`#include "ImgWindow/contrib/widgets/imgui_mouse_wheel_controls.h"`	Include only the specific control family you need.
Opt-Out Macros	`#define CONTRIB_DISABLE_<FEATURE>`	Zero conflict if your project already defines a conflicting identifier.

Opt-Out Configuration Flags

If your project defines a function with the same name, define any of the following canonical macros *before* including `contrib.h` or `widgets.h` (`IMGUI_CONTRIB_DISABLE_*`, `IMGUI_DISABLE_CONTRIB_*`, and legacy `IMGUI_DISABLE_EXTRA_*` macro names are also honored automatically):

```[CPP]
#define CONTRIB_DISABLE_TOGGLE_BUTTONS        // Disables ToggleButton &amp;amp; ToggleButtons
#define CONTRIB_DISABLE_TOGGLE_MOUSE_WHEEL    // Disables mouse-wheel support on ToggleButton &amp;amp; ToggleButtons
#define CONTRIB_DISABLE_MOUSE_WHEEL_CONTROLS  // Disables all wheel sliders, drags, cyclers &amp;amp; combos
#define CONTRIB_DISABLE_CLICKABLE_LINKS       // Disables ButtonLink, TextLink &amp;amp; AttachItemCursor
#define CONTRIB_DISABLE_BADGES                // Disables Badge, Pill &amp;amp; Kbd widgets
#define CONTRIB_DISABLE_SPINNERS              // Disables Spinner, RadialTimer &amp;amp; CountdownTimer

// Fine-grained opt-outs &amp;amp; style defaults within imgui_mouse_wheel_controls.h:
#define CONTRIB_DISABLE_GET_MOUSE_WHEEL       // Disables GetMouseWheel* primitives
#define CONTRIB_DISABLE_ADJUST_MOUSE_WHEEL    // Disables AdjustItemOnMouseWheel()
#define CONTRIB_DISABLE_CYCLE_MOUSE_WHEEL     // Disables CycleItemOnMouseWheel()
#define CONTRIB_DISABLE_PERCENT_SLIDER        // Disables PercentSlider() &amp;amp; SliderPercent()
#define CONTRIB_DISABLE_SLIDER_PERCENT        // Disables only the SliderPercent() alias
#define CONTRIB_DISABLE_MOUSE_WHEEL_COMBO     // Disables MouseWheelCombo &amp;amp; ComboWithReset
#define IMGUI_CONTRIB_DISABLE_RESET_UNDO      // Disables 1-click Reset-with-Undo (restore previous value)

// Optional compile-time icon &amp;amp; chevron style customization:
#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL      ICON_FA_UNDO     // Custom primary reset icon (default: vect
#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT  ICON_FA_REDO     // Custom restore/redo icon (default: vecto
#define IMGUI_CONTRIB_DEFAULT_COMBO_ARROW_STYLE ImGui::ComboArrowStyle_RightWhenClosed // Combo chevron pol
```

Widget Catalog

1. Toggle Controls (`imgui_toggle_buttons.h`)

A. Animated Toggle Switch (`ImGui::ToggleButton`)

A modern, iOS / Material-design rounded pill switch with a smoothly sliding circular thumb. Accepts both `bool*` and `int*` state variables.

```cpp
#include "contrib/widgets/imgui_toggle_buttons.h"

static bool s_ExpertMode = false;
static int  s_SmoothTransitions = 1;

// 1. Standard switch with clickable label on the right:
ImGui::ToggleButton("Expert Mode", &amp;s_ExpertMode);

// 2. Direct reference syntax (no &amp; required):
ImGui::ToggleButton("Expert Mode", s_ExpertMode);

// 3. Compact switch without visible label (uses ID only):
ImGui::ToggleButton("##hud_switch", &amp;s_HudActive);

// 4. Direct dual-string per-state tooltips (Off tooltip, On tooltip):
ImGui::ToggleButton("HUD Overlay", &amp;s_HudActive,
                    "HUD is currently **OFF**.\n- Click to enable overlay.",
                    "HUD is currently **ON**.\n- Click to disable overlay.");

// 5. Array-based per-state tooltips ({ Off tooltip, On tooltip }):

static const char* const kTransitionTooltips[] = {
    "Smooth transitions are **OFF** (instant snap).\n- Click or scroll **up** on the switch to enable.",
    "Smooth transitions are **ON** (animated).\n- Click or scroll **down** on the switch to disable."
};
ImGui::ToggleButton("Smooth Transitions", &amp;s_SmoothTransitions, ImVec2(0, 0), kTransitionTooltips);

// Optional: Customize switch active track &amp; thumb colors globally:
ImGui::SetDefaultToggleButtonColors(ImVec4(0.26f, 0.59f, 0.98f, 1.0f)); // Active track color
// Or via ImU32:
ImGui::SetDefaultToggleButtonColors(ABCIM_COL32(HILITE_COLOR));
```

- **Features:**
- Frame-rate independent sliding animation powered by `io.DeltaTime` and stored per-widget in `ImGuiStorage`.
- **Flexible Types & Syntax:** Accepts `bool*`, `int*`, `bool&`, or `int&` (`0` = Off, non-zero = On).
- **Global Theme Integration (`SetDefaultToggleButtonColors`):** Easily set the switch track/thumb colors globally via `ImVec4` or `ImU32`. Defaults to your theme's active highlight color (`ImGuiCol_ButtonActive`).
- **Built-In Clamped Mouse-Wheel Support (`allow_mouse_wheel = true`):** Spinning the scroll wheel up/right over the pill switch turns it **On** and clamps there; spinning down/left turns it **Off** and clamps there (never wraps). Hovering over the text label does *not* capture the mouse wheel, preventing accidental toggles when scrolling windows.
- **Per-State Tooltips:** Supports both direct dual-string parameters (`off_tooltip`, `on_tooltip`) and 2-element string arrays (`{ off_tooltip, on_tooltip }`) automatically rendered via `TimedTooltip::TextMDProxy` (or standard `SetTooltip` fallback).
- **Responsive Auto-Scaling:** Switch dimensions scale naturally with the ambient font size (0.78x ambient frame height for standard switches, 0.60x for compact switches), looking crisp on standard, Retina, and 4K displays, with support for explicit `ImVec2 size` overrides.
- Automatic hand pointer cursor (`ImGuiMouseCursor_Hand` by default, customizable via `hover_cursor` or `IMGUI_EXTRA_TOGGLE_DEFAULT_CURSOR`).

B. Segmented Dual-Button Pair (`ImGui::ToggleButtons`)

Joined side-by-side buttons for explicit binary selections (`Off` / `On`, `Solid Opaque` / `Glass Translucent`). Accepts both `int*` and `bool*`.

```
[CPP]
static int s_WindowStyle = 0; // 0 = Solid Opaque, 1 = Glass Translucent

// 1. Compact pair without a right-side label (starts with "##"):
if (ImGui::ToggleButtons("##style_toggle", &amp;s_WindowStyle, "Solid Opaque", "Glass Translucent")) {
    ApplyStyleChange();
}

// 2. With clickable right-side title ("Surface Style") and wheel-latched hover index:
int hovered_idx = -1;
ImGui::ToggleButtons("Surface Style##style_pair", &amp;s_WindowStyle, "Solid Opaque", "Glass Translucent",
                     ImVec2(130.0f, 0.0f), nullptr /* item_tooltips */, &amp;hovered_idx);
```

- **Features:**
- **Optional Right-Side Clickable Title:** Following standard Dear ImGui `"Title##id"` semantics, if `label` contains text before `"##"`, it is drawn to the right of the button pair, shows a Hand cursor on hover, toggles the state on click, and ignores the mouse wheel.
- **Clamped Mouse-Wheel Toggling:** Scrolling up/right over either button selects `1` (`label_on`); scrolling down/left selects `0` (`label_off`).
- **Per-Button Tooltips (`item_tooltips`) & Wheel-Selection Tooltip Latch (`out_hovered_item`):** When the user spins the mouse wheel over either button to change the state, tooltip targeting (and `*out_hovered_item`) automatically latches onto the newly selected button (`0` or `1`)—even if the stationary mouse cursor is still parked over the unselected button—until the mouse moves (`> 1.5px`).

---

2. Mouse-Wheel Sliders, Drag Boxes, Cyclers & Combos (`imgui_mouse_wheel_controls.h`)

A. Core Mouse-Wheel Primitives (`ImGui::GetMouseWheel` / `GetMouseWheelV` / `GetMouseWheelH` / `GetMouseWheelEither`)

Queries vertical (`GetMouseWheel` / `GetMouseWheelV`), horizontal (`GetMouseWheelH`), or either-axis (`GetMouseWheelEither`, alias `GetMouseWheelAny`) scroll-wheel deltas for the current frame. Passing `consume = true` zeroes out `io.MouseWheel` / `io.MouseWheelH` so parent scrollable windows do not scroll while adjusting a hovered control.

```
[CPP]
float clicks = ImGui::GetMouseWheelEither(true /* consume event */);
```

B. Continuous Range Chaser (`ImGui::AdjustItemOnMouseWheel`)

Attaches to **any** preceding continuous range item (`SliderFloat`, `SliderInt`, `DragFloat`, `DragInt`, etc.) via `ImGui::IsItemHovered()`. Overloaded for both `float*` and `int*`, it interprets vertical/horizontal mouse-wheel clicks, consumes the wheel event, and clamps boundaries.

- **Track-Only Cursor Precision:** Switches the cursor to East-West resize arrows (`ImGuiMouseCursor_ResizeEW` by default) strictly when hovering over the physical control track/box, preserving standard arrow cursors over text labels.

```cpp
#include "contrib/widgets/imgui_mouse_wheel_controls.h"

// Attach mouse-wheel adjustment to any standard ImGui slider or drag box:
ImGui::SliderFloat("Text Zoom", &amp;fontZoom, 0.75f, 1.75f);
ImGui::AdjustItemOnMouseWheel(&amp;fontZoom, 0.75f, 1.75f, 0.01f);

ImGui::DragInt("Altitude", &amp;alt, 10.0f, 0, 40000);
ImGui::AdjustItemOnMouseWheel(&amp;alt, 0, 40000, 100.0f);
```

C. Discrete Choice Chaser (`ImGui::CycleItemOnMouseWheel`)

Attaches to **any** preceding widget (`Button`, `Text`, `Combo`, `Selectable`) to step an integer index forward or backward one item per scroll gesture. Overloaded for both `(int* idx, int item_count, ...)` and `(int* idx, int min_idx, int max_idx, ...)`.

- **Clamped by Default (`wrap = false`):** Stops cleanly at the first and last items so a fast flick of the scroll wheel never accidentally wraps from minimum to maximum.
- **Directional Hover Cursor:** Defaults to `ImGuiMouseCursor_ResizeNS` (↕) on hover, and can be overridden per-call (e.g., `ImGuiMouseCursor_ResizeEW` ↔ for horizontal button cycling or `ImGuiMouseCursor_Hand`).

```cpp
// 1. Attach wheel-cycling to a popup-menu opener button with an East-West scroll cursor:
if (ImGui::ButtonLink(kDensityLabels[s_DensityIdx])) {
    ImGui::OpenPopup("##density_menu");
}
if (ImGui::CycleItemOnMouseWheel(&amp;s_DensityIdx, 5 /* item_count */, false /* wrap */,
                                 true /* natural_list_order */, true /* consume */,
                                 ImGuiMouseCursor_ResizeEW)) {
    ApplyDensity(s_DensityIdx);
}

// 2. Standalone wheel-cycled ButtonLink paired with an abutting ResetButton:
//    Evaluating `(is_mod || can_restore) &amp;&amp; static_cast&lt;bool&gt;(btn)` resets/restores on click
//    when actionable, and automatically falls back to click-to-toggle tooltip when idle at default!
const bool is_mod      = (s_DensityIdx != 2);
const bool can_restore = (!is_mod &amp;&amp; s_LastDensityIdx != 2);
auto btn = ImGui::ButtonLink(kDensityLabels[s_DensityIdx], ImVec2(110.0f, 0.0f));
bool btn_clicked = (is_mod || can_restore) &amp;&amp; static_cast&lt;bool&gt;(btn);
if (ImGui::CycleItemOnMouseWheel(&amp;s_DensityIdx, 5, false, true, true, ImGuiMouseCursor_ResizeEW)) {
    if (s_DensityIdx == 2) s_LastDensityIdx = 2;
}

ImGui::SameLine(0.0f, 0.0f);
bool rst_clicked = ImGui::ResetButton("##density_reset", is_mod, 0.0f, true /* abutting */, nullptr, can_restore);
if (btn_clicked || rst_clicked) {
    if (is_mod) { s_LastDensityIdx = s_DensityIdx; s_DensityIdx = 2; }
    else if (can_restore) { s_DensityIdx = s_LastDensityIdx; }
    ImGui::TimedTooltip::ClearActiveTooltip();
}
```

D. Turnkey Mouse-Wheel Sliders & Drag Boxes (`MouseWheelSlider` / `PercentSlider` / `MouseWheelDrag`)

Drop-in replacements for standard sliders and drag boxes that respond to mouse-wheel scrolling out of the box. `MouseWheelSlider` and `MouseWheelDrag` are overloaded for both `float*` and `int*` so you never have to type `Float` or `Int` in the function name!

```cpp
static float s_Heading = 180.0f;
static int   s_Fov     = 80;
ImGui::MouseWheelSlider("Heading", &amp;s_Heading, 0.0f, 360.0f, "%.0f°", 0, 1.0f);
ImGui::MouseWheelSlider("FOV",     &amp;s_Fov,     40,   120,    "%d°",   0, 1.0f);

static float s_PitchOffset = 0.0f;
ImGui::MouseWheelDrag("Pitch Offset", &amp;s_PitchOffset, 0.5f /* speed &amp; default wheel step */, -45.0f
```

E. Composite Sliders & Drag Boxes with Reset & Undo (`SliderWithReset` / `PercentSliderWithReset` / `DragWithReset`)

Full-featured compound controls (overloaded for `float*` and `int*`, also aliased as `ResettableSlider`, `ResettablePercentSlider`, and `ResettableDrag`) with built-in mouse-wheel support, a flush abutting vector undo/reset button (`0`-gap seam), smart width reservation, hover isolation, active input lockout, and **Reset-with-Undo** (1-click A/B toggle between default and previous value).

- Sits cleanly in a single line: `[ Control Track/Box | ■ / ■ ] Label Text`
- **"Reset with Undo" (1-Click A/B Comparison):**
- When modified (`*v != v_default`), clicking the Reset button saves the current non-default value (`last_val`), resets `*v = v_default`, and flips the button into **Restore/Redo Mode** (■ or `IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT`).
- Clicking the button again in Restore mode swaps `*v` right back to `last_val`, letting users click back and forth effortlessly to A/B compare their custom setting against the default!
- **Automatic Session Memory + Optional Persistent Pointer (`v_last`):** Works automatically with **zero extra arguments** via internal per-ID session memory (`##reset_undo`), or accepts an optional trailing `float* v_last = nullptr` / `int* v_last = nullptr` pointer if you want to persist the pre-reset value in your own variable/dataref across sessions.
- **Automatic 3-State Default Tooltips:** When `reset_tooltip` is `nullptr`, automatically formats the default and previous values using the control's `format` string:
- **Modified:** `"Click to revert to default (85%)."`
- **After Reset (Restore Mode):** `"At default (85%). Click to restore previous (42%)."`
- **At Default (Idle):** `"Already at default (85%)."` (Clicking the dimmed button or right-side label immediately toggles the status/help `TimedTooltip` ON/OFF!)
- **Dual Icon Configuration (`#define` or Runtime Setter) & Vector Fallback:**
- With zero font configuration, draws crisp, DPI-scaled procedural vector arrows directly in `ImDrawList`: counter-clockwise ■ for Reset, and clockwise ■ for Restore-Previous.
- Or configure custom FontAwesome / icon-font strings at compile time (`#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL ICON_FA_UNDO` and `#define IMGUI_CONTRIB_DEFAULT_RESET_LABEL_ALT ICON_FA_REDO` / `IMGUI_CONTRIB_DEFAULT_REDO_LABEL`) or at runtime via `ImGui::SetDefaultResetButtonLabel(reset_label, restore_label)`. Button width is automatically sized across both glyphs so the control never shifts by 1px when toggling states.
- **Seamless Abutting Frame Borders:** When abutting an active control (`abutting == true`), `ResetButton` automatically suppresses redundant internal frame borders (`FrameBorderSize > 0.0f`), ensuring the control track and reset button share a single clean outer frame without an unwanted vertical box seam between them.

```cpp
static float s_Heading = 180.0f;
static int   s_Fov     = 80;

// Optional: configure custom Reset &amp;amp; Restore/Redo icons globally (or via #define before #include):
// ImGui::SetDefaultResetButtonLabel(ICON_FA_UNDO, ICON_FA_REDO);

// 1-line sliders (float* or int*) with built-in mouse-wheel and Reset-with-Undo:
ImGui::SliderWithReset("Heading", &amp;amp;s_Heading, 180.0f /* default */, 0.0f, 360.0f, "%.0f°");
ImGui::SliderWithReset("FOV",     &amp;amp;s_Fov,     80     /* default */, 40,   120,    "%d°");

static float s_Opacity = 0.85f;
ImGui::PercentSliderWithReset("Opacity", &amp;amp;s_Opacity, 0.85f /* default */, 0.1f, 1.0f);

static float s_PitchOffset = 0.0f;
ImGui::DragWithReset("Pitch Offset", &amp;amp;s_PitchOffset, 0.0f /* default */, 0.5f /* speed */, -45.0f, 45.0
```

F. Turnkey Mouse-Wheel Combos (`MouseWheelCombo` / `ComboWithReset`)

Dropdown combo boxes that work like standard `ImGui::Combo` when clicked, and support live mouse-wheel cycling both when **closed** and while the dropdown popup menu is **open**!

- **Split Preview vs. Chevron `[▼]` Zones:**
- **Preview Text Zone (`[box_min.x .. box_max.x - GetFrameHeight()]`):** Shows `hover_cursor` (`ImGuiMouseCursor_ResizeNS` by default) and cycles items on mouse-wheel without falsely highlighting the right-hand chevron button.
- **Chevron Button Zone (`[box_max.x - GetFrameHeight() .. box_max.x]`):** Shows `ImGuiMouseCursor_Hand`, highlights strictly when the chevron itself is hovered (or when the popup is open), and **ignores mouse-wheel scrolling** so clicking/hovering the opener button feels like a dedicated dropdown button.
- **Configurable Chevron Arrow Polarity (`ImGui::ComboArrowStyle`):**

Configure globally via `#define IMGUI_CONTRIB_DEFAULT_COMBO_ARROW_STYLE` or `ImGui::SetDefaultComboArrowStyle(...)`:

- `ImGui::ComboArrowStyle_DownWhenClosed` (`0`, default): ▼ when closed $\rightarrow$ ▲ when open (or ▼ if the popup flipped upward near the bottom of the screen).
- `ImGui::ComboArrowStyle_RightWhenClosed` (`1`, ABC disclosure style): ■ when closed $\rightarrow$ ▼ (or ▲ if opened upward) when open.
- `ImGui::ComboArrowStyle_UpWhenClosed` (`2`): ▲ when closed $\rightarrow$ ▼ (or ▲ if opened upward) when open.
- `ImGui::ComboArrowStyle_StaticDown` (`3`): Vanilla ImGui's static ▼.
- `ComboWithReset` (also aliased as `ResettableCombo`) pairs the mouse-wheel combo with the flush abutting ■ / ■ reset-with-undo button (with automatic session memory or optional `int* last_item = nullptr` persistence).
- **Per-Item Tooltips (`item_tooltips[]`) & Open-Menu Wheel Tooltip Latch ("Lens UI"):** Pass an optional array of tooltip strings (plain text or Markdown when `contrib.h` is active) to attach a distinct tooltip to each item in the open dropdown list. When the user spins the mouse wheel while the dropdown menu is open, tooltip targeting automatically latches onto the **newly wheel-selected item** (`*current_item`) rather than whichever row the stationary mouse cursor happens to be parked over, and seamlessly returns to normal mouse hover targeting when the mouse moves (`> 6px`).

```cpp
static const char* const kCameraModes[] = { "Cockpit", "Chase", "Circle", "Tower", "Ride-Along" };
static const char* const kCameraModeTooltips[] = {
    "**Cockpit View**: Inside the 3-D flight deck.",
    "**Chase View**: Locked behind the aircraft.",
    "**Circle View**: Free exterior orbit around the aircraft.",
    "**Tower View**: Stationary camera from the nearest ATC tower.",
    "**Ride-Along**: Wing-mount or fuselage camera position."
};
static int s_CamMode = 1; // Default: Chase (1)

// Optional: use disclosure chevron polarity (■ when closed -&gt; ▼ when open):
// ImGui::SetDefaultComboArrowStyle(ImGui::ComboArrowStyle_RightWhenClosed);

// Standard mouse-wheel-enabled dropdown with per-item tooltips:
ImGui::MouseWheelCombo("Camera View", &amp;s_CamMode, kCameraModes, IM_ARRAYSIZE(kCameraModes),
                       -1, false /* wrap */, kCameraModeTooltips);

// Compound mouse-wheel dropdown with abutting Reset-with-Undo (1 = Chase):
ImGui::ComboWithReset("Camera Mode", &amp;s_CamMode, 1 /* default */,
                      kCameraModes, IM_ARRAYSIZE(kCameraModes),

                      -1, false /* wrap */, nullptr /* auto 3-state reset/restore tooltip */,
                      kCameraModeTooltips);
```

G. Mouse-Wheel Checkboxes (`MouseWheelCheckbox` / `ToggleItemOnMouseWheel` / `CycleItemOnMouseWheel(bool*)`)

Allows standard checkboxes to be toggled with the mouse wheel strictly when hovering over the **square `[✓]` box on the left** (`[box_min.x .. box_min.x + GetFrameHeight()]`), while hovering over the **text label** shows `ImGuiMouseCursor_Hand` (still clickable to toggle) and **ignores** the mouse wheel!

- By default (`wrap = false`), scrolling **up/right** checks the box (`true`) and scrolling **down/left** unchecks it (`false`), clamping cleanly at both ends (or pass `wrap = true` to flip on every wheel notch).
- Includes convenience overloads accepting `ImGuiMouseCursor_` directly after `bool* v` so you can customize or suppress the hover cursor (`ImGuiMouseCursor_Hand` or `ImGuiMouseCursor_None`) without passing intermediate boolean parameters.

```cpp
static bool s_AutoZoom = true;

// Option 1: Single-call MouseWheelCheckbox (with optional cursor override right after &amp;v):
ImGui::MouseWheelCheckbox("Enable Auto-Zoom", &amp;s_AutoZoom);
ImGui::MouseWheelCheckbox("Quiet Mode", &amp;s_QuietMode, ImGuiMouseCursor_Hand);

// Option 2: 1-line chaser attached to standard ImGui::Checkbox:
ImGui::Checkbox("Expert Mode", &amp;s_ExpertMode);
ImGui::ToggleItemOnMouseWheel(&amp;s_ExpertMode, ImGuiMouseCursor_None); // or CycleItemOnMouseWheel(&amp;s_ExpertMode)
```

3. Clickable Link & Item Cursor Affordances (`imgui_clickable_links.h`)

In standard Dear ImGui, buttons, menu items, and clickable text keep the default OS arrow cursor. The `imgui_clickable_links.h` header eliminates the boilerplate of writing `if (IsItemHovered()) SetMouseCursor(ImGuiMouseCursor_Hand);` on every interactive item, while providing automatic **click-to-toggle tooltip** ergonomics and **open-popup wheel-selection tooltip latching**.

- **Ambient Color Preservation on `ImGui::TextLink`:** In Dear ImGui 1.91+, the built-in `ImGui::TextLink(const char*)` forces `ImGuiCol_TextLink` (a bright sky-blue tint) with an underline. Our `imgui_clickable_links.h` overloads preserve your **ambient `ImGuiCol_Text` color** by default (or accept an explicit `const ImVec4* col_override` and `std::string`), while rendering a zero-frame clickable text item with a Hand cursor on hover (with Dear ImGui's native hyperlink preserved as `ImGui::NativeTextLink`).

- **Smart Click-to-Toggle Tooltips (`ImGui::LinkClickResult`):** `ButtonLink`, `SmallButtonLink`, `TextLink`, `TextLinkUnformatted`, `MenuItemLink`, and `MD::TextLink` return `ImGui::LinkClickResult`:
- **Tested in `if (...)` (Action Link):** Evaluating `if (ImGui::ButtonLink("Save"))` invokes `operator bool()`, marking the click as handled by your action block and dismissing any open tooltip on click.
- **Standalone Statement (Info / Help Link):** Calling `ImGui::TextLink("(i) Help");` without testing its return value leaves the click unconsumed, so clicking the widget immediately toggles a trailing `ImGui::TimedTooltip::Text` / `TextMD` ON and OFF (without closing any enclosing `BeginPopup`!).

```cpp
#include "contrib/widgets/imgui_clickable_links.h"

// 1. Action button (tested in `if` -&gt; executes action &amp; dismisses tooltip on click):
if (ImGui::ButtonLink("Save Flight Plan")) {
    SavePlan();
}
ImGui::TimedTooltip::TextMD("Saves the active flight plan to disk.");

// 2. Standalone info link (return value ignored -&gt; clicking toggles tooltip ON/OFF!):
ImGui::TextLinkUnformatted("(i) Works with scroll-wheel!");
ImGui::TimedTooltip::TextMD("Spin the **scroll-wheel** over this menu to step through options.");

// 3. Checkboxes, radio buttons, selectables, and menu items with hand cursor:
ImGui::CheckboxLink("Enable Autothrottle", &amp;s_Autothrottle);
ImGui::RadioButtonLink("NAV 1", &amp;s_NavSource, 1);
ImGui::SelectableLink("Row Item", isSelected);
ImGui::MenuItemLink("Compact Density", nullptr, isSelected);
```

Custom `BeginPopup` Menus with Mouse-Wheel Cycling & Per-Item Tooltips

When building custom `ImGui::BeginPopup` menus (with custom titles, separators, or footers), wrap the popup contents in `BeginGroup()` / `EndGroup()`, use `ImGui::MenuItem` (or `ImGui::MenuItemLink` for a Hand cursor on each row) with per-item `TimedTooltip::TextMD` calls, and attach `ImGui::CycleItemOnMouseWheel` after `EndGroup()`:

```cpp
if (ImGui::BeginPopup("#density_menu")) {
    ImGui::BeginGroup();
    if (ImGui::TextLink("Choose tracking list density:")) {
        ImGui::CloseCurrentPopup();
    }
    ImGui::Separator();

    for (int i = 4; i &gt;= 0; --i) {
        bool isActive = (densityIdx == i);
        if (ImGui::MenuItem(menus[i], nullptr, isActive)) {
            gUserLinePadding = 4 + 2 * i;
        }
        ImGui::TimedTooltip::TextMD("Switch table spacing to **%s** density.", tipLabels[i]);
    }

    ImGui::Separator();
    ImGui::TextLinkUnformatted("  (i) Works with scroll-wheel!  ");
    ImGui::TimedTooltip::TextMD("Spin your scroll-wheel over this list to step through options.");
    ImGui::EndGroup();

    // Whole-menu mouse-wheel cycling (automatically latches per-item tooltips to `isActive`!):
    if (ImGui::CycleItemOnMouseWheel(&amp;densityIdx, 0, 4, /*wrap=*/false, /*natural_list_order=*/false,
                                     /*consume=*/true, ImGuiMouseCursor_Hand)) {
        gUserLinePadding = 4 + 2 * densityIdx;
    }
    ImGui::EndPopup();
}
```

- **Automatic Wheel-Selection Tooltip Latch:** `ImGui::MenuItem` and `ImGui::MenuItemLink` register each item's `isActive` state. When the user spins the scroll wheel inside the open popup, `TimedTooltip` automatically switches

from mouse-hover targeting to showing the tooltip for the **newly wheel-selected item** (`isActive`), returning to normal hover targeting as soon as the mouse moves (`> 6px`).

Universal Item Hover-Cursor Chaser (`ImGui::AttachItemCursor`):

```cpp
// Attach default Hand cursor to any preceding item:
ImGui::Image(textureId, ImVec2(64, 64));
ImGui::AttachItemCursor();

// Or attach any specific ImGuiMouseCursor on hover:
ImGui::Button("Splitter");
ImGui::AttachItemCursor(ImGuiMouseCursor_ResizeEW);
```

4. Badges, Pills & Key Caps (`imgui_badges.h`)

Sleek, standalone C++ UI indicators designed for status tags, version numbers, notification counters, and keyboard shortcut chips without requiring any Markdown parsing.

```cpp
#include "contrib/widgets/imgui_badges.h"

// 1. Semantic Status Badges (Default, Primary, Success, Warning, Danger, Info):
ImGui::Badge("ONLINE", ImGui::ImGuiBadgeVariant_Success);
ImGui::SameLine();
ImGui::Badge("EXPIRING", ImGui::ImGuiBadgeVariant_Warning);
ImGui::SameLine();
ImGui::Badge("FAULT", ImGui::ImGuiBadgeVariant_Danger);

// 2. Fully-rounded Pills:
ImGui::Pill("v1.92", ImGui::ImGuiBadgeVariant_Primary);
ImGui::SameLine();
ImGui::Pill("PRO", ImVec4(0.85f, 0.40f, 0.10f, 1.0f)); // Custom ImVec4 background

// 3. Custom Hex or ImU32 Backgrounds with Automatic Contrast Calculation:
ImGui::Badge("BETA", ImGui::GetColorU32(ImVec4(0.9f, 0.9f, 0.1f, 1.0f))); // Auto-selects dark text

// 4. Keyboard Key Caps (&lt;kbd&gt;):
ImGui::Text("Press");
ImGui::SameLine();

ImGui::Kbd("Ctrl+Shift+P");
ImGui::SameLine();
ImGui::Text("to open Command Palette.");
```

- **Features:**
- **Pixel-Balanced Rounded Geometry:** Uses TrueType baseline alignment and font-scale compensation so ascenders and descenders ('p', 'g', 'y') stay centered with identical visual margins.
- **Intelligent Auto-Contrast:** Computes ITU-R BT.601 perceptual luminance of the background color and automatically selects crisp dark (`#000000`) or light (`#FFFFFF`) text when foreground color is omitted.
- **3D Key Caps (`ImGui::Kbd`):** Renders tactile keyboard shortcut chips with rounded borders and a bottom bevel shadow.
- **Zero Markdown Overhead:** Standalone immediate-mode C++ functions usable anywhere in pure ImGui panels. Markdown's `<badge>`, `<pill>`, and `<kbd>` tags delegate directly to these widgets.

5. Procedural Spinners & Countdown Dials (`imgui_spinners.h`)

Smooth, procedural immediate-mode activity indicators and radial timers rendered directly via `ImDrawList` antialiased paths. No texture assets, no font hacks, and zero memory allocations.

```cpp
#include "contrib/widgets/imgui_spinners.h"

// 1. Organic Activity Spinner (loading / busy state):
ImGui::Spinner("##loading"); // Auto-scales to font size
ImGui::SameLine();
ImGui::Text("Syncing telemetry...");

// With custom radius, thickness, color, and label:
ImGui::Spinner("Downloading airport navdata", 14.0f, 3.0f, IM_COL32(0, 200, 255, 255));

// 2. Radial Countdown Timer (fraction remaining: 0.0f .. 1.0f):
float timeoutRemaining = GetRemainingFraction(); // e.g., 0.65f
ImGui::RadialTimer("Auto-dismiss in 5s", timeoutRemaining);

// 3. Countdown Timer Helper (current seconds vs max duration):
float alertTimeLeft = 3.2f;
float alertMaxTime  = 10.0f;
ImGui::CountdownTimer("Alert Timeout", alertTimeLeft, alertMaxTime);
```

- **Features:**
- **Organic Breathing Arc Animation:** Rather than a rigid rotating stick, **`ImGui::Spinner`** uses a sinusoidal expansion-and-contraction cycle that mimics modern Material/Apple loading spinners.
- **Clockwise/Counter-Clockwise Dials:** **`ImGui::RadialTimer`** starts cleanly at 12 o'clock (**`-90°`**) with an ambient background track ring, sweeping smoothly around to indicate elapsed or remaining time.
- **True Ambient Scaling:** Radius and stroke thickness automatically scale to the ambient font size (**`ImGui::GetFontSize()`**) when left as **`0.0f`**, looking sharp on standard and high-DPI displays alike.
- **Built-in Text Label Affordance:** Labels without leading `##` automatically render beside the spinner or dial with standard item alignment and spacing.

Credits & Upstream Lineage

- **Author & Architect**: Steven L. Goldberg (@slgoldberg)
- **SliderPercent Upstream Inspiration**: Birger Hoppe (LiveTraffic)
- **License**: BSD 3-Clause (see ImgWindow LICENSE)

Markdown Extension

The **Markdown** extension provides rich Markdown parsing and rendering for Dear ImGui within ImgWindow, based on **`enkisoftware/imgui_markdown`** and heavily enhanced for X-Plane plugin user interfaces.

It allows you to embed rich text formatting, headers, links, lists, and custom interactive widgets directly into your ImGui windows, settings panels, and stationary tooltips.

> **Author / Maintainer:** Steven L. Goldberg (@slgoldberg)

> **Original Upstream:** Juliette Foucaut (@juliettef) & Doug Binks (@dougbinks) — Repository: enkisoftware/imgui_markdown

> **License:** BSD 3-Clause (Extension Wrapper; see ImgWindow LICENSE) & zlib (Upstream Parser)

Architectural Overview: Foundation vs. Extension Wrapper

The Markdown system is structured as a clean two-layer architecture:

1. **Base Engine (`imgui_markdown.h`):**

- Upstream parser originally created by Doug Binks and Juliette Foucaut (**`enkisoftware/imgui_markdown`**).
- Responsible for raw character scanning, CommonMark tokenization, AST-like block dispatch, and low-level C-style callbacks (**`formatCallback`**, **`linkCallback`**, **`imageCallback`**, **`tooltipCallback`**).
- Our fork incorporates crucial low-level fixes for CommonMark punctuation lookahead and bullet list token isolation.

2. **ImgWindow Extension Wrapper (`imgui_markdown_ext.h`):**

- High-level, ergonomic C++ layer designed specifically for X-Plane plugins and modern ImGui applications.
- Encapsulates all functionality inside the second-level **`ImGui::Markdown::`** namespace, with **`ImGui::MD::`** provided as an official, zero-overhead namespace alias (**`namespace ImGui { namespace MD = Markdown; }`**).
- Eliminates the upstream symbol collision: Upstream's raw C-style function **`ImGui::Markdown(text, len, config)`** has been moved to **`ImGui::Markdown::Render(...)`** (and **`ImGui::MD::Render(...)`**). This prevents beginners from accidentally invoking the raw unconfigured parser and frees **`ImGui::Markdown`** to be a clean, first-class C++ namespace.
- Introduces an XML/HTML-style inline tag pre-processor (**`<tag>`**, **`<color>`**, **`<backdrop>`**, **`<badge>`**, **`<btn>`**) with custom tag callback registration via **`ImGui::Markdown::RegisterTag`**.
- Solves Dear ImGui layout hurdles: typographic line-wrap lookahead (eliminating mid-word breaks across styles and links), mathematical hanging indents for wrapped multi-line bullets, and ambient font scale inheritance (**`ImGui::GetFontScale()`**).
- Provides non-rendering layout measurement via **`ImGui::Markdown::CalcTextSize`** (defeating ImGui viewport culling to measure exact dimensions for floating windows and stationary tooltips).
- Directly interfaces with the companion **`TimedTooltip`** extension via bidirectional bridges (**`ImGui::TimedTooltip::Markdown::Text`** / **`ImGui::TimedTooltip::MD::Text`** and **`ImGui::Markdown::TimedTooltip::Text`** / **`ImGui::MD::TimedTooltip::Text`**).

Architectural Enhancements over Upstream

- **Collision-Free Canonical Namespace (`ImGui::Markdown::` & `ImGui::MD::`):**
- Primary namespace is **`ImGui::Markdown::Text(...)`** for full clarity.
- Shorthand alias **`ImGui::MD::Text(...)`** is 100% interchangeable.
- Root convenience aliases **`ImGui::TextMD(...)`** / **`ImGui::TextMarkdown(...)`** allow drop-in usage matching **`ImGui::Text(...)`**.

- Opt-in global convenience (`#define CONTRIB_ENABLE_GLOBAL_NAMESPACES` or `#include "contrib/contrib-global.h"`) publishes `Markdown::Text(...)` and `MD::Text(...)` to the global scope for developers seeking minimum boilerplate.
- **CommonMark Backslash Escaping:**
- Escapes any ASCII punctuation character (`\*`, `\#`, `\_`, `\<`, `\>`, `\[`, `\]`, `\(`, `\)`, `\\`, etc.) to render literally without triggering emphasis, headers, HTML tags, or links.
- Ensures clean display of technical expressions like `C\*` or `[X-Plane 12]\(build\)`.
- **Strikethrough & Underline Typography:**
- **Strikethrough:** Full support for GitHub Flavored Markdown `~~deleted text~~` and legacy ABC `$$struck text$$`, as well as HTML tags (`<s>`, `<del>`, `<strike>`).
- **Underline:** Supports CriticMarkup / Markdown Extended `++underlined text++` and HTML tags (`<u>`, `<ins>`) out-of-the-box. Setting `mdConfig.underlineWithUnderscores = true` allows Discord/ABC-style `__underlined text__` to render as underline instead of bold.
- **Pixel-Aligned Rules:** Centered midline for strikethrough and baseline-aligned underline scale cleanly across all display DPIs and custom font sizes.
- **Ordered & Unordered Lists with True Hanging Indents:**
- **Ordered Lists:** Automatically recognizes numbered items (`1.` , `2.` , etc.) and formats the number prefix with a clean hanging indent matching unordered bullets.
- **Unordered Lists:** Replaces cramped upstream bullet spacing with scalable, comfortable breathing room (`bulletSpacing` defaulting to a clean 4px).
- Mathematically maintains true hanging indents across wrapped multi-line list items and continuation lines, keeping all wrapped text cleanly aligned to the text column rather than under the bullet glyph or number.
- Preserves leading space depth across emphasis chunks and links so nested sub-bullets never pop out of alignment.
- Broadens list marker syntax to standard Markdown (`*`, `-`, `+`) with or without leading whitespace, with strict token isolation so bullet glyphs never hijack inline styling into rogue italic runs.
- **Layout Pre-Calculation (`CalcSize` / `CalcTextSize`):**
- Exposes `ImGui::MD::CalcSize(...)` (aliased as `CalcTextSize`) to pre-measure exact wrapped multi-line Markdown dimensions without rendering to the screen—crucial for sizing OS floating windows, child frames, and stationary tooltips.
- Defeats ImGui viewport culling by employing pass-unique offscreen window hashes.
- **Dynamic Font Scaling:**
- Uses `ImGui::GetFontScale()` to cleanly honor parent window text zoom and high-DPI display scales.
- **Custom Tag Registration:**
- Supports custom XML/HTML-style tags (e.g., `<btn>Label</btn>`, `<badge>OK</badge>`) that execute arbitrary C++ rendering callbacks inline with text flow.
- **Collision-Free Scope (`ImGui::MD::`):**
- Scoped under `ImGui::MD::` to mirror standard ImGui vocabulary (`Text`, `TextWrapped`) without colliding with core ImGui functions.

Quick Start

1. Include the Header

```cpp
#include "contrib/extensions/Markdown/imgui_markdown_ext.h"
// (Or simply: #include "contrib.h")
```

IMPORTANT

Migrating from Upstream `imgui_markdown`: Remove Old Includes! If your plugin previously included upstream `imgui_markdown.h` directly: 1. **Remove your old `#include "imgui_markdown.h"`** lines and delete any loose copies of `imgui_markdown.h` from your project's header search paths. 2. **Include `#include "contrib.h"` (or `#include "contrib/extensions/Markdown/imgui_markdown_ext.h"`) instead.** `imgui_markdown_ext.h` automatically includes our modernized, patched engine (`contrib/extensions/Markdown/imgui_markdown.h`) under the hood. Keeping an older upstream copy in your project tree risks compiling against the unpatched upstream engine, which lacks our typographic lookahead, mathematical hanging indents, and `ImGui::MD::` widget suite!

2. Configure Default Styles & Fonts (Optional)

You are responsible for loading your own fonts into your `ImgFontAtlas` during plugin startup (e.g., in `InitializeImGui()`). There is no runtime font synthesization in Dear ImGui—if you want Markdown to render distinct headers, bold text, or italics, you must load those fonts into your atlas.

As demonstrated in `sample-code/InitializeImGui.cpp`, each sequential, non-merged font added increments an index in `ImGui::GetIO().Fonts->Fonts[i]`. We recommend defining macro aliases for these slots:

```cpp
// Example font slot definitions (from sample-code/InitializeImGui.cpp):
#define IM_FONT_SLOT(i)       ImGui::GetIO().Fonts-&gt;Fonts[i]
#define IM_NORMAL_FONT        IM_FONT_SLOT(0) // Regular base font (14px)
#define IMG_TITLE_FONT        IM_FONT_SLOT(1) // Large header font
#define IM_SMALLER_FONT       IM_FONT_SLOT(2) // Compact UI font
#define IM_BOLD_FONT          IM_FONT_SLOT(3) // Bold font
#define IM_BOLD_LARGER_FONT   IM_FONT_SLOT(4) // Bold title font
#define IM_MONO_NORMAL_FONT   IM_FONT_SLOT(5) // Monospace code font
#define IM_ITALIC_FONT        IM_FONT_SLOT(6) // Italic font
```

During plugin startup, configure your Markdown appearance globally using `ImGui::MD::Config`:

```cpp
ImGui::MD::Config mdConfig;

// Configure semantic fonts directly (no cryptic array indices!):
mdConfig.h1Font     = IM_BOLD_LARGER_FONT; // # H1
mdConfig.h2Font     = IMG_TITLE_FONT;      // ## H2
mdConfig.h3Font     = IM_BOLD_FONT;        // ### H3
mdConfig.boldFont   = IM_BOLD_FONT;        // **Bold** / __Bold__
mdConfig.italicFont = IM_ITALIC_FONT;      // *Italic* / _Italic_
mdConfig.monoFont   = IM_MONO_NORMAL_FONT; // `code` / &lt;code&gt; / &lt;mono&gt;

// Configure tight, clean bullet spacing:
mdConfig.bulletSpacing = 4.0f;

// Set globally for the extension:
ImGui::SetDefaultMarkdownConfig(&amp;mdConfig);
```

NOTE

Graceful Font Fallback: If any font pointer is left as `nullptr`, the engine gracefully falls back: * `h1Font`, `h2Font`, `h3Font` fall back to `headingFormats[0..2]` (or the ambient window font). * `boldFont` falls back to `headingFormats[3]` (or the ambient window font). * `italicFont` falls back to `ImGuiCol_TextDisabled` (upstream's dimmed text color). * `monoFont` falls back to the ambient window font.

3. Rendering Markdown in Windows

Simply call `ImGui::Markdown::Text` (or the shorthand `ImGui::MD::Text`):

```cpp
// Standard single-line or multi-line Markdown:
ImGui::Markdown::Text("### System Status\nEngine 1: **Normal** | Fuel: *Sufficient*");

// Paragraph block with automatic word wrapping:
ImGui::Markdown::TextWrapped(
    "Check this option to enable **custom cursors**. "
    "Changes take effect immediately without requiring a simulator restart."
);

// Clickable Markdown text link:
if (ImGui::Markdown::TextLink("Open **Avionics Settings**")) {
    OpenSettings();
}
```

TIP

Forgiving Aliases: Prefer shorter names? `ImGui::MD::Text(...)` is an official shorthand alias for `ImGui::Markdown::Text(...)`. Root shorthands like `ImGui::TextMD(...)` work directly under `ImGui::` as well.

TIP

`#`, `##`, and `###` Are Pure Markdown Headers (Never Swallowed as ImGui IDs!): In standard Dear ImGui widgets (`ImGui::Button`, `ImGui::Checkbox`, `ImGui::CollapsingHeader`, etc.), `"##"` is reserved as an ID separator that hides everything after `"##"`. In `ImGui::MD::Text(...)`, `ImGui::MD::TextWrapped(...)`, and `ImGui::TimedTooltip::TextMD(...)`, text is rendered via `ImGui::TextUnformatted(start, end)`, bypassing ImGui's `"##"` label parser completely. You can freely start strings or lines with `#`, `##`, or `###` for H1/H2/H3 headers without any escaping!

4. Rendering Markdown in Timed Tooltips

When combined with the companion **TimedTooltip** extension (included in either order, or via `#include "contrib/includes.h"`), rich multi-line Markdown tooltips are attached with a single line of code:

```cpp
#include "contrib/includes.h"

ImGui::Button("Accessibility Mode");
ImGui::TimedTooltip::TextMD(
    "### Accessibility Options\n"
    "Enables **larger fonts** and high-contrast styling across all plugin chrome.\n\n"
    "* Fast toggle: `Ctrl+Shift+A`\n"
    "* Requires zero reload."
);
```

Configuration Reference (`ImGui::MD::Config` / `ImGui::MarkdownConfig`)

For clean namespace symmetry with `ImGui::TimedTooltip::Config`, the configuration struct is accessible as both `ImGui::MD::Config` and `ImGui::MarkdownConfig`.

Field	Type	Default	Description
`boldFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for bold emphasis (`**bold**` / `__bold__`). (Falls back to `headingFormats[3].font`.)

Field	Type	Default	Description
`italicFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for true italics (`*italic*` / `_italic_`). (Falls back to `ImGuiCol_TextDisabled` if null.)
`monoFont`	`ImFont*`	`nullptr`	**[Semantic]** Font used for monospace code spans (` code `) and **[mono]** / **[code]**` tags. (Falls back to ambient font if null.)
`h1Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H1 headers (`# Header`). (Falls back to `headingFormats[0].font`.)
`h2Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H2 headers (`## Header`). (Falls back to `headingFormats[1].font`.)
`h3Font`	`ImFont*`	`nullptr`	**[Semantic]** Font used for H3 headers (`### Header`). (Falls back to `headingFormats[2].font`.)
`bulletSpacing`	`float`	`0.0f` (`4.0f` active)	Horizontal gap in pixels between the bullet circle glyph and the following text.
`headingFormats`	`HeadingFormat[4]`	`{}`	Upstream array fallback for H1 (`[0]`), H2 (`[1]`), H3 (`[2]`), and Bold (`[3]`).
`linkCallback`	`LinkCallback`	`nullptr`	Upstream hook invoked when the user clicks a markdown `[link](url)`.
`tooltipCallback`	`TooltipCallback`	`nullptr`	Upstream hook invoked when hovering a link with tooltip description.
`imageCallback`	`ImageCallback`	`nullptr`	Upstream hook invoked to render inline markdown `![alt](url)` images.
`formatCallback`	`FormatCallback`	`defaultMarkdownFormatCallback`	Low-level upstream hook invoked when entering/exiting format blocks (headers, lists, bold, italic, code).

NOTE

Do I Need to Implement the Callbacks? In 99% of UI tasks, **no**. The callbacks (`linkCallback`, `tooltipCallback`, `imageCallback`, and `formatCallback`) are low-level C-style hooks inherited directly from the upstream `enkisoftware/imgui_markdown` base engine. With the semantic font properties (`boldFont`, `italicFont`, `monoFont`, `h1Font`, etc.) and the high-level tag registration API (`ImGui::MD::RegisterTag`), you do not need to write custom format callbacks.

Emphasis & Monospace Typography: Bold, Italics, and Code

Standard Markdown and CommonMark recognize distinct emphasis and code levels:

- **Level 1 Emphasis (`*italic*` or `_italic_`):** Traditionally rendered as italics (`<em>`).
- **Level 2 Emphasis (`**bold**` or `__bold__`):** Traditionally rendered as bold (`<strong>`).
- **Code Spans (` code ` / **[code]** / **[mono]**):** Rendered in monospace font (**[code]**`).

1. Bold Emphasis (`**bold**` / `__bold__`)

Bold emphasis automatically uses `mdConfig.boldFont`. If set (e.g. `IM_BOLD_FONT`), all double-delimiter text renders in crisp bold. If left `nullptr`, it checks `headingFormats[3].font` before falling back to the ambient window font.

2. True Italics (`*italic*` / `_italic_`)

In upstream `enkisoftware/imgui_markdown`, there was no field for italics, so single-delimiter text was given a shortcut: tinted with `ImGuiCol_TextDisabled` (a dimmed, half-opacity gray).

With our extension, simply assign your loaded italic font to `mdConfig.italicFont`:

```
[CPP]
mdConfig.italicFont = IM_ITALIC_FONT;
```

When `italicFont` is assigned, `*hello*` and `_hello_` render in true, crisp italic typography rather than dimmed disabled gray. If left `nullptr`, it preserves upstream's disabled-color fallback for compatibility.

3. Monospace Code Spans (` code `) and Code Tags ([code] / [mono]`)

For code identifiers, hotkeys, file paths, and terminal syntax, assign your loaded monospace font to `mdConfig.monoFont`:

```
[CPP]
mdConfig.monoFont = IM_MONO_NORMAL_FONT;
```

You have two complementary options for rendering monospace text:

- **Inline Backtick Spans (` code `):** Standard CommonMark inline backticks switch cleanly to mdConfig.monoFont` while preserving the surrounding text color and sentence flow.
- **Code Tags (`<code>foobar</code>` or `<mono>foobar</mono>`):** Switches to `mdConfig.monoFont` and automatically draws a subtle, translucent rounded pill background (`IM_COL32(128, 128, 128, 60)`) behind the text bounding box, providing a sleek styled code badge (similar to GitHub and documentation sites). Accepts optional background tinting via parameter (e.g. `<code=#334455>foobar</code>`).

4. Multi-Line Fenced Code Blocks (` ```lang / ~~~lang `)

For multi-line snippets, function signatures, configuration files, and terminal logs, use standard CommonMark triple backticks (or tildes):

```

void Initialize() {

ImGui::Badge("ONLINE", ImGui::ImGuiBadgeVariant_Success);

}

```

- **Features:**
- **Automatic Monospace Typography:** Uses `mdConfig.monoFont` (or ambient monospace font) line-by-line without word-wrapping or accidental markup parsing.
- **Shaded Framed Box:** Renders an attractive, rounded charcoal background box with a subtle border using two-pass draw list channels (`ChannelsSplit`).
- **Language Chip Indicator:** Displays the language identifier (e.g., `cpp`, `lua`, `python`, `json`, `plaintext`) as a neat badge chip in the upper corner of the block.
- **CommonMark Compliant:** Supports both backticks (` ``` `) and tildes (~~~`) with optional indentation and variable fence lengths.

Per-Call Configuration Overrides

Every rendering function in `ImGui::MD` accepts an optional `const MarkdownConfig*` pointer if a specific widget requires unique fonts, callbacks, or bullet spacing:

```
[CPP]

ImGui::MD::Config customConfig = *ImGui::GetDefaultMarkdownConfig();
customConfig.bulletSpacing = 6.0f;

ImGui::MD::Text(markdownString, &amp;customConfig);
```

Layout Pre-Calculation (`CalcTextSize`)

When building stationary tooltips or sizing dynamic OS windows to fit a block of documentation, you need to know the rendered height *before* drawing:

```
[CPP]

ImGuiID widgetId = ImGui::GetID("doc_section");
float wrapWidth = 400.0f; // Constrain to 400px wide
float fontScale = 0.0f;   // 0.0f = inherit ambient scale

// Pre-calculates exact ImVec2 dimensions without rendering to the screen:
ImVec2 requiredSize = ImGui::MD::CalcTextSize(widgetId, 1, docString, wrapWidth, fontScale);

// Use the size to configure window bounds or child frames:
ImGui::SetNextWindowSize(requiredSize);
```

Markdown Styling Tags (`<tag=param>` / `<tag param>`)

The extension includes a built-in pre-processor that allows you to embed rich, inline styling tags and interactive widgets directly within Markdown text without breaking standard Markdown formatting rules:

```
[MARKDOWN]

This option is &lt;color=red&gt;highly experimental&lt;/color&gt;!
Please verify &lt;backdrop=yellow&gt;active route&lt;/backdrop&gt; before flight.
System status: &lt;badge=danger&gt;FAULT&lt;/badge&gt; | &lt;badge=success&gt;ONLINE&lt;/badge&gt;
```

1. Flexible Tag Syntax

Tags support both HTML-style attributes and key-value formats:

- **Assignment format:** `<tag=red>text</tag>`, `<backdrop=yellow>text</backdrop>`
- **Space-separated format:** `<tag red>text</tag>`, `<color #00FFCC>text</color>`
- **Multi-parameter format:** `<tag=red,bold>text</tag>`, `<tag red bold>text</tag>`

2. Bold, Italic, and Monospace Styling Inside Tags

You can style tag text in two natural ways:

A. Markdown Syntax Inside the Tag (Recommended)

Place standard Markdown delimiters directly inside the tag content:

```
[MARKDOWN]
Check &lt;tag red&gt;**accessibility**&lt;/tag&gt; or options.
Please review &lt;backdrop=yellow&gt;**critical checklist**&lt;/backdrop&gt; before departure.
Click &lt;btn&gt;*more info*&lt;/btn&gt; for details.
&lt;pill orange&gt;` ■ `&lt;/pill&gt;
```

- The engine detects inner Markdown wrappers (**`**bold**`**, **`__bold__`**, **`*italic*`**, **`_italic_`**, ` mono **`), strips the wrapper characters so delimiters never render literally, and pushes the active bold, italic, or monospace (`**mdConfig.monoFont`) font.
- Intentional leading/trailing padding spaces (both outside and inside the delimiters, as well as UTF-8 non-breaking spaces **`U+00A0`**) are preserved so you can easily pad single-icon pills (e.g., **`"<pill orange mono> " ICON_FA_RETWEET " </pill>"`** or **`"<pill orange>`** " ICON_FA_RETWEET " **`</pill>"`**).
- For **`<backdrop>`** and **`<pill>`**, the font is pushed *before* measuring text width, ensuring the pill bounding box fits the styled glyphs.

B. Tag Parameter Flags

You can also specify **`bold`** (or **`b`**), **`italic`** (or **`i`**), and **`mono`** (or **`monospace`**, **`code`**, **`m`**) directly in the tag attributes (alongside optional **`color=`** / **`col=`** prefixes):

```
[MARKDOWN]
Check &lt;tag red bold&gt;accessibility&lt;/tag&gt; or options.
Check &lt;tag=red,bold&gt;accessibility&lt;/tag&gt; or options.
&lt;pill orange mono&gt; ■ &lt;/pill&gt;
&lt;pill color=0xFFA500, mono&gt; ■ &lt;/pill&gt;
&lt;btn bold&gt;Acknowledge&lt;/btn&gt;
```

> **NOTE**
>
> **Why `**<tag>...</tag>**` is not supported:** In Markdown and CommonMark, formatting delimiters wrap text spans rather than embedding interactive widgets across boundary tokens. Nesting formatting inside the tag (**`<tag>**text**</tag>`**) or on the tag (**`<tag bold>text</tag>`**) follows standard HTML/XML element structure and guarantees correct word wrapping and draw-list grouping.

3. Inline Spacing & Punctuation Flow

Inline tags integrate cleanly into paragraph and sentence flow:

- **Preserved Spaces:** Both inter-word spaces after a closing tag (such as **`<tag red>badge</tag> or options`**) and intentional padding spaces inside a tag (such as **`<pill orange mono> ■ </pill>`**) are preserved.
- **Flush Punctuation:** When followed by punctuation without space, such as **`<tag red>badge</tag>, options`**, the punctuation docks directly flush against the tag border.
- **Automatic Line-Wrap Protection:** If an inline tag, button, or background pill exceeds the remaining horizontal space on the current line, the renderer automatically wraps down to the next line (**`EnsureTagFitsOnLine`**) to prevent tags from spilling past window borders.

4. Pre-Supplied Built-in Tags (Ready Out-of-the-Box)

The following tags are pre-registered and active by default:

Tag	Syntax Options	Parameters & Variants	Description
color / **col**	**<color=COLOR>text</color>** **<color COLOR [bold] [italic] [mono]>text</color>**	Named colors (**red**, **green**, **blue**, **yellow**, **orange**, **purple**, **violet**, **cyan**, **magenta**, **white**, **black**, **gray**, **gold**, **dark**, **light**) or hex (**#RRGGBB**, **#RRGGBBAA**, **0xRRGGBB**, **0xRRGGBBAA**). Optional flags: **bold**, **italic**, **mono**.	Renders **text** in the specified color and font style.
backdrop / **bg** / **highlight**	**<backdrop=COLOR>text</backdrop>** **<backdrop COLOR [bold] [italic] [mono]>text</backdrop>**	Accepts any named color or hex code. Optional attribute flags: **bold**, **italic**, **mono**.	Renders a rounded background rectangle ("pill") optically centered behind the text bounding box. Backdrops auto-contrast text color against dark backgrounds and never split awkwardly across lines.
badge / **pill** / **tag**	**<badge=VARIANT>text</badge>** **<pill COLOR [bold] [italic] [mono]>text</pill>** **<pill bg=BG fg=FG>text</pill>**	Semantic variants (**danger**, **warning**, **success**, **info**), named colors, or custom hex color. Supports explicit dual-color attributes (**bg=COLOR fg=COLOR**, **color=COLOR text=COLOR**). Optional flags: **bold**, **italic**, **mono**.	Delegates directly to **ImGui::Badge** / **ImGui::Pill** standalone C++ widgets. Renders sleek rounded badges with auto-contrasting text.
kbd / **key**	**<kbd>Key</kbd>** **<key>Ctrl+C</key>**	Keyboard key or shortcut label.	Delegates directly to **ImGui::Kbd**. Renders a 3D tactile keyboard keycap with bottom bevel/shadow.
u / **ins**	**<u>text</u>** **<ins>text</ins>**	Optional color attribute (**<u #FFCC00>**).	Renders text with a pixel-aligned underline rule along the baseline.
s / **del** / **strike**	**<s>text</s>** **~~text~~** **<strike>text</strike>**	Optional color attribute (**<s red>**).	Renders text with a pixel-aligned strikethrough rule across the vertical midline.
btn	**<btn=ID>label</btn>** **<btn [bold] [italic] [mono]>label</btn>**	Unique button ID or style flags (**bold**, **italic**, **mono**).	Renders an inline, clickable **ImGui::SmallButton** directly within text flow.
mono / **code**	**<mono>text</mono>** **<code [COLOR]>text</code>**	Accepts optional custom background color or hex parameter (and optional **fg=COLOR**).	Renders **text** in monospace (**mdConfig.monoFont**) inside a subtle, translucent rounded background pill.

*(To disable default tags, define **#define IMGUI_DISABLE_MARKDOWN_DEFAULT_TAGS** before including the header.)*

Two-Color Pills & Badges (**bg=** and **fg=**)

By default, **<badge>**, **<pill>**, and **<backdrop>** automatically contrast the text color against the background (e.g., using dark text on yellow, light text on dark blue). When you want explicit artistic control over both colors, specify **bg** (background) and **fg** (text/foreground):

```
[MARKDOWN]
Status: &lt;pill bg=warning fg=black&gt;CAUTION&lt;/pill&gt;
Release: &lt;badge bg=#1a3a5a fg=#40c0ff bold&gt;v2.2.1&lt;/badge&gt;
Dual-tone: &lt;pill purple white italic&gt;SPECIAL&lt;/pill&gt;
```

Inner Markdown vs. Tag Attributes

It's helpful to distinguish how formatting reaches your tags:

1. **Inner Markdown (<tag>**text**</tag>):**

Works universally across **all** tags (both built-in tags and your own custom registered tags). If you write standard Markdown delimiters inside a tag, the engine automatically pushes the matching bold/italic font and strips the delimiter characters *before* calling the tag callback.

2. **Tag Attributes (`<tag param>text</tag>`):**

Built-in tags (**`badge`**, **`backdrop`**, etc.) inspect the opening tag's attribute string for colors and layout flags. When writing custom tags, the raw parameter string is passed directly into your callback as the second argument (**`param`**), allowing you to build arbitrary domain-specific markup (e.g., **`<item id=42 count=3>Widget</item>`**).

---

5. Custom Tag Registration (`ImGui::MD::RegisterTag`)

You can register your own custom styling tags or widgets using **`ImGui::MD::RegisterTag`** (or **`ImGui::RegisterMarkdownTag`**) with either 1-argument or 2-argument (parameterized) callbacks:

```cpp
// 1. Parameterized tag: &lt;status=critical&gt;Engine 1&lt;/status&gt; or &lt;status ok&gt;Engine 2&lt;/status&gt;
ImGui::MD::RegisterTag("status", [](const std::string&amp; text, const std::string&amp; param) {
    ImVec4 col = (param == "critical") ? ImVec4(1.0f, 0.2f, 0.2f, 1.0f) : ImVec4(0.2f, 0.8f, 0.2f, 1.0f);
    ImGui::PushStyleColor(ImGuiCol_Text, col);
    ImGui::TextUnformatted(text.c_str());
    ImGui::PopStyleColor();
});

// 2. Simple single-argument tag: &lt;notice&gt;Check checklists&lt;/notice&gt;
ImGui::MD::RegisterTag("notice", [](const std::string&amp; text) {
    ImGui::BulletText("%s", text.c_str());
});
```

> **TIP**
>
> **A Friendly Note on C++ Lambdas:** If you are new to modern C++, the `[](const std::string& text, const std::string& param) { ... }` syntax is standard C++11 lambda notation for an inline anonymous function. The lambda receives whatever text was enclosed between `` and `` as `text`, and any parameters or attributes from the opening tag as `param`.

*(Note: **`ImGui::RegisterMarkdownTag`** and **`ImGui::RegisterMarkdownWidget`** are also preserved as direct backward-compatible aliases.)*

API Summary

```cpp
namespace ImGui {

    // Global Configuration &amp;amp; Tag Registry:
    void SetDefaultMarkdownConfig(const MarkdownConfig* config);
    const MarkdownConfig*&amp;amp; GetDefaultMarkdownConfig();
    void RegisterMarkdownTag(const std::string&amp;amp; tag_name, TagCallback callback);
    void RegisterMarkdownWidget(const std::string&amp;amp; tag_name, CustomTagCallback callback);

    namespace MD {
        // Configuration Alias:
        using Config = MarkdownConfig;

        // Custom Tag Registration (Convenience Aliases):
        void RegisterTag(const std::string&amp;amp; tag_name, std::function&amp;lt;void(const std::string&amp;amp; text
        void RegisterTag(const std::string&amp;amp; tag_name, std::function&amp;lt;void(const std::string&amp;amp; text

        // Standard Text Rendering (Mirroring standard ImGui vocabulary):
        // - TextUnformatted / Text: Do not wrap by default (unless wrap_width / wrap_size or PushTextWrapP
        // - TextWrapped: Wraps automatically to GetContentRegionAvail().x (or explicit wrap_width / wrap_s
        void TextUnformatted(const char* text, const char* text_end = nullptr, const MarkdownConfig* config

        void TextUnformatted(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_override =
        void Text(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_override = nullptr);
        void Text(const std::string&amp;amp; markdown_text, float wrap_width, const MarkdownConfig* config_over
        void Text(const std::string&amp;amp; markdown_text, const ImVec2&amp;amp; wrap_size, const MarkdownConfig*
        void TextWrapped(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_override = null
        void TextWrapped(const std::string&amp;amp; markdown_text, float wrap_width, const MarkdownConfig* conf
        void TextWrapped(const std::string&amp;amp; markdown_text, const ImVec2&amp;amp; wrap_size, const MarkdownC

        // Clickable Markdown Text Links (Hand cursor on hover; toggles TimedTooltip on click if return val
        LinkClickResult TextUnformattedLink(const char* text, const char* text_end = nullptr, const Markdow
        LinkClickResult TextUnformattedLink(const std::string&amp;amp; markdown_text, const MarkdownConfig* con
        LinkClickResult TextLink(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_overrid
        LinkClickResult TextLink(const std::string&amp;amp; markdown_text, float wrap_width, const MarkdownConf
        LinkClickResult TextLink(const std::string&amp;amp; markdown_text, const ImVec2&amp;amp; wrap_size, const M
        LinkClickResult TextWrappedLink(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_
        LinkClickResult TextWrappedLink(const std::string&amp;amp; markdown_text, float wrap_width, const Markd
        LinkClickResult TextWrappedLink(const std::string&amp;amp; markdown_text, const ImVec2&amp;amp; wrap_size,

        // Backward-Compatibility Aliases:
        void TextMD(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_override = nullptr);

        void TextWrappedMD(const std::string&amp;amp; markdown_text, const MarkdownConfig* config_override = nu

        // Layout Pre-Measurement:
        ImVec2 CalcSize(ImGuiID id, int pass, const std::string&amp;amp; text, float wrap_width = 0.0f, float f
        ImVec2 CalcTextSize(ImGuiID id, int pass, const std::string&amp;amp; text, float wrap_width = 0.0f, flo
        ImVec2 CalcMarkdownSize(ImGuiID id, int pass, const std::string&amp;amp; text, float wrap_width = 0.0f,

        // Interactive Button with Tooltip:
        bool ButtonMD(const std::string&amp;amp; label, const std::string&amp;amp; tooltip_md = "", float tooltip_w
    }
}
```

Appendix: Real-World Example & Comparison

Below is a complete real-world example illustrating how the **Markdown** and **TimedTooltip** extensions combine to create a rich, responsive settings and reference panel with custom tags, nested list items, and stationary tooltips.

Side-by-Side: Why Use the Extensions?

To appreciate what the extensions handle behind the scenes, consider what is required to build a mixed-format settings card with highlight backdrops, colored status pills, bullet lists with hanging indents, and a stationary clamped tooltip:

The Hard Way: Vanilla Dear ImGui (Without Extensions)

[CPP]

```
// ----------------------------------------------------------------------------
// VANILLA IMGUI: ~70+ lines of low-level boilerplate, manual geometry math,
// and state storage just for a single header, backdrop pill, and timed tooltip!
// ----------------------------------------------------------------------------
void RenderSettingsPanelVanilla()
{
    ImGui::Begin("Plugin Settings &amp; Reference");

    // 1. Header: Manual font push, separator, and manual pop
    ImGui::PushFont(IMG_TITLE_FONT);
    ImGui::TextUnformatted("Welcome to Avionics Manager");
    ImGui::PopFont();
    ImGui::Separator();

    // 2. Mixed Bold Text + Background Highlight Pill:
    // In vanilla ImGui, you cannot wrap text across lines if you embed custom drawn rectangles.
    // You have to manually calculate draw list coordinates, text sizes, and padding:
    ImGui::TextUnformatted("Configure your autopilot, navigation modes, and ");
    ImGui::SameLine(0.0f, 0.0f);

    ImVec2 p0 = ImGui::GetCursorScreenPos();
    ImVec2 text_sz = IM_BOLD_FONT-&gt;CalcTextSizeA(IM_BOLD_FONT-&gt;FontSize, FLT_MAX, 0.0f, "expert features");
    ImVec2 p1 = ImVec2(p0.x + text_sz.x + 8.0f, p0.y + text_sz.y + 2.0f);

    // Draw background pill manually:
    ImGui::GetWindowDrawList()-&gt;AddRectFilled(p0, p1, IM_COL32(255, 230, 60, 255), 4.0f);

    // Position text inside pill and draw in dark contrast color:
    ImGui::SetCursorScreenPos(ImVec2(p0.x + 4.0f, p0.y + 1.0f));
    ImGui::PushFont(IM_BOLD_FONT);
    ImGui::PushStyleColor(ImGuiCol_Text, IM_COL32(20, 20, 20, 255));
    ImGui::TextUnformatted("expert features");
    ImGui::PopStyleColor();
    ImGui::PopFont();

    // Resume following text:
    ImGui::SetCursorScreenPos(ImVec2(p1.x, p0.y));
    ImGui::TextUnformatted(" below.");

    ImGui::Spacing();

    // 3. Bullet List:
    // Vanilla ImGui::BulletText does NOT support hanging indents when text wraps.
    // Continuation lines wrap directly beneath the bullet circle instead of indenting cleanly.
    // Fixing this requires manual ImGui::Indent() / ImGui::Unindent() bookkeeping.
    ImGui::Bullet();
    ImGui::SameLine(0.0f, 4.0f);
    ImGui::TextWrapped("autopilot: Controls automated flight guidance (LNAV, VNAV, FLCH).");

    ImGui::Spacing();

    // 4. Reset Button with Stationary Clamped Tooltip:
    // Vanilla ImGui::SetTooltip moves with the mouse, has no hover delay, and lacks screen clamping.
    // Building a stationary tooltip with a 1-second delay and viewport boundary clamping requires
    // querying frame counters, ImGuiStorage, and calculating display boundary math:
    ImGui::Button("Reset to Defaults");

    static int s_HoverFrames = 0;
    if (ImGui::IsItemHovered()) {
        s_HoverFrames++;
```

```
        if (s_HoverFrames &gt; 60) { // ~1.0 second delay
            ImVec2 mousePos = ImGui::GetIO().MousePos;
            ImVec2 dispSize = ImGui::GetIO().DisplaySize;
            ImVec2 tipSize(260.0f, 80.0f); // Guessed height!

            // Manual screen boundary clamping:
            ImVec2 tipPos = ImVec2(mousePos.x, mousePos.y + 20.0f);
            if (tipPos.y + tipSize.y &gt; dispSize.y) tipPos.y = mousePos.y - tipSize.y - 10.0f;
            if (tipPos.x + tipSize.x &gt; dispSize.x) tipPos.x = dispSize.x - tipSize.x - 10.0f;

            ImGui::SetNextWindowPos(tipPos);
            ImGui::PushStyleColor(ImGuiCol_PopupBg, IM_COL32(255, 242, 153, 255));
            ImGui::PushStyleColor(ImGuiCol_Text, IM_COL32(0, 0, 0, 255));
            if (ImGui::BeginTooltip()) {
                ImGui::TextUnformatted("Restores all settings to factory defaults.");
                ImGui::EndTooltip();
            }
            ImGui::PopStyleColor(2);
        }
    } else {

        s_HoverFrames = 0;
    }

    ImGui::End();
}
```

The Clean Way: With ImgWindow Extensions (`MD` + `TimedTooltip`)

[CPP]

```
// ---------------------------------------------------------------------------
// WITH EXTENSIONS: Clean, declarative, expressive code.
// The engine automatically handles dynamic font scaling, lookahead word wrapping,
// hanging bullet indents, draw-list pills, delay timers, and screen docking!
// ---------------------------------------------------------------------------
#include "contrib/extensions/Markdown/imgui_markdown_ext.h"
#include "contrib/extensions/TimedTooltip/imgui_tooltip_ext.h"

void RenderSettingsPanelExtensions()
{
    ImGui::Begin("Plugin Settings &amp; Reference");

    // 1. Rich Header and Overview with Custom Backdrop Pill and Tags:
    ImGui::MD::TextWrapped(
        "### Welcome to Avionics Manager\n"
        "Configure your autopilot, navigation modes, and &lt;backdrop=yellow&gt;**expert features**&lt;/backdrop&gt; help
        "Hover over any setting or &lt;tag red&gt;tag&lt;/tag&gt; for operational manuals."
    );

    ImGui::Separator();
```

```
    // 2. Multi-Line List with Nested Bullets, Badges, and Automatic Hanging Indents:
    ImGui::MD::TextWrapped(
        "* &lt;tag red&gt;modes&lt;/tag&gt;: _Click_ on active tags above to filter the feature catalog.\n"
        "  * _Important_: Filter queries use exact \"&lt;tag yellow&gt;substring&lt;/tag&gt;\" matching.\n"
        "  * Toggle back to preset: `__#1__` / \"**Default Nav**\" &lt;pill green&gt;Active&lt;/pill&gt;.\n
        "* &lt;tag blue&gt;autopilot&lt;/tag&gt;: Controls automated lateral and vertical flight guidance.\
        "  * Supported modes: **LNAV**, **VNAV**, and **FLCH**.\n"
        "  * Disconnect shortcut: `Ctrl+Shift+D`."
    );

    ImGui::Spacing();

    // 3. Interactive Widgets with Stationary Clamped Markdown Timed Tooltip:
    if (ImGui::Button("Reset to Defaults")) {
        // Reset logic...
    }
    // Automatically binds to the preceding button, manages hover cycle delays,
    // isolates styles, clamps to screen borders, and renders rich Markdown:
    ImGui::TimedTooltip::TextMD(

        "### Reset Configuration\n"
        "Restores all plugin settings back to factory defaults.\n\n"
        "* **Reversible:** A backup of your current setup is saved automatically.\n"
        "* Active theme: #3/\"**Black &amp; White**\" &lt;pill green&gt;PG&lt;/pill&gt;.\n"
        "* *Click to confirm.*"
    );

    ImGui::End();
}
```

TimedTooltip Extension

The **TimedTooltip** extension provides an advanced, stationary tooltip engine designed specifically for Dear ImGui within X-Plane plugins.

Unlike standard Dear ImGui tooltips which move with the mouse and can flicker, obstruct UI elements, or run off-screen, `TimedTooltip` locks in place, calculates dynamic viewport boundaries, preserves custom interactive cursors, isolates its styling hermetically from parent windows, and supports both plain-text and rich Markdown content.

> **Author / Maintainer:** Steven L. Goldberg (@slgoldberg)

> **License:** BSD 3-Clause (see ImgWindow LICENSE)

Key Features

- **Configurable Delay & Lifetime:**
- **Low Watermark (Hover Delay):** Prevents tooltip spam by waiting for the user to pause over a widget before showing.
- **Active Input Low-Water Reset:** Spinning the mouse wheel (`io.MouseWheel` / `io.MouseWheelH`) or dragging (`IsMouseDragging`) over any widget immediately slams the hover timer back to 0 and dismisses any active tooltip, preventing tooltips from popping up or obstructing adjustments during continuous wheel scrolling or slider interaction.
- **High Watermark (Auto-Dismiss):** Automatically hides tooltips after a readable duration, with adaptive time-scaling for taller/multi-line text.
- **Grace Period:** Forgives micro-slips off the target widget so users don't lose their place.
- **Wiggle Latch:** Micro-mouse movements reset the auto-hide timer so users can continue reading long descriptions.
- **Hermetic Style Isolation:**
- Pushes and pops its own `WindowPadding`, `ItemSpacing`, `FramePadding`, and `IndentSpacing`. Outer parent window style modifications (such as `ItemSpacing.x = 0` in menu bars or compressed button pads) can never distort tooltip typography, and tooltip styles never leak back out.
- Precise vertical line pitch (`item_spacing.y = 0.0f`) preserves the exact 15px native font leading in X-Plane, preventing vertical bloat and screen overrun.
- **Smart Bounds Clamping & Docking:**
- **Cursor-Safe Viewport Clamping (Flip Above Cursor):** If a tooltip would clip off the bottom of the screen (e.g. over a bottom control strip or footer slider), it automatically flips **above the mouse cursor**, ensuring the tooltip never covers the cursor or the widget being manipulated.
- **Leftward Shift:** If a tooltip approaches the right screen border, it automatically shifts leftwards.
- **Adaptive Wrap:** Shrink-wraps tightly around concise text while wrapping paragraphs at a readable width (80% of available viewport width by default).
- **Interactive Cursor Protection & Click-to-Toggle Ergonomics:**
- Preserves and restores custom widget cursors (e.g., `ImGuiMouseCursor_Hand` or `ImGuiMouseCursor_ResizeEW`), and automatically shows a Hand cursor on passive text/labels that have a tooltip attached.
- Clicking passive text, right-side Slider/Drag/Combo labels, dimmed at-default reset buttons, or standalone `*Link` widgets immediately toggles the tooltip ON/OFF without waiting for `hover_delay_cycles`.
- **Multi-`ImGuiContext` & Overlapping-Widget Safety:**
- Isolates tooltip state and wheel-selection latches per `ImGuiContext*` so multiple `ImgWindow` instances rendering in the same X-Plane draw cycle never interfere with one another.
- **Foreground-Widget Hover Priority:** When a button sits on the same line as a full-width background `Selectable(..., ImGuiSelectableFlags_SpanAllColumns)`, `TimedTooltip` automatically gives tooltip priority to the earlier foreground widget on the row without requiring manual hover-suppression flags.

- **Plain Text & Markdown Support (Order-Independent):**
- Native auto-wrapping plain-text via **`Text(...)`**, single-line non-wrapping plain-text via **`TextUnformatted(...)`**, rich auto-wrapping Markdown via **`TextMD(...)`**, and single-line non-wrapping Markdown via **`TextUnformattedMD(...)`**.
- **`imgui_tooltip_ext.h`** and **`imgui_markdown_ext.h`** can be **`#include`**d in **either order** and the **`TextMD`** / **`TextUnformattedMD`** bridge is synthesized automatically.

Quick Start

1. Include the Header

```cpp
#include "contrib/extensions/TimedTooltip/imgui_tooltip_ext.h"
```

2. Basic Usage (Plain Text)

Attach a timed tooltip directly after drawing any ImGui widget:

```cpp
if (ImGui::Button("Save Flight Plan")) {
    SavePlan();
}
// Automatically attaches to the preceding widget:
ImGui::TimedTooltip::Text("Saves the active route to disk as an .fms file.");
```

3. Markdown Tooltips

If using the companion **Markdown** extension, simply call **`TextMD`**:

```cpp
#include "contrib.h"

ImGui::Button("Autopilot Nav");
ImGui::TimedTooltip::TextMD("Arms **LNAV** mode. *Requires active GPS waypoint.*");
```

> **TIP**
>
> **Forgiving Aliases:** Prefer explicit sub-namespaces? **`ImGui::TimedTooltip::Markdown::Text(...)`** and **`ImGui::TimedTooltip::MD::Text(...)`** work too! If you prefer flipping the order (**`ImGui::Markdown::TimedTooltip::Text`**), that works identically. Clickable links inside tooltips are supported via **`TextMDLink(...)`**.

Configuration (`ImGui::TimedTooltip::Config`)

The **`TooltipConfig`** (aliased as **`ImGui::TimedTooltip::Config`**) controls timing, dimensions, typography, and colors.

Configuration Fields Reference

Field	Type	Default	Description
`hover_delay_cycles`	`int`	`65`	Frame cycles the mouse must hover before the tooltip appears (~1.0s at 60fps).

Field	Type	Default	Description
`max_display_cycles`	`int`	`504`	Frame cycles before the tooltip auto-dismisses. Set negative to disable auto-hide.
`grace_period_cycles`	`int`	`12`	Allowable frame cycles off the widget before cancelling hover state.
`size_skew_multiplier`	`float`	`0.5f`	Extra cycles added to display timeout per pixel of vertical height.
`enable_wiggle_latch`	`bool`	`true`	When `true`, subtle mouse movements inside the tooltip reset the dismiss timer.
`default_wrap_width`	`float`	`360.0f`	Target comfortable sticky-note width (~50-60 chars). Short text shrink-wraps tightly to its natural size.
`min_wrap_width`	`float`	`240.0f`	Minimum width boundary before forced truncation/overflow.
`max_wrap_width`	`float`	`600.0f`	Maximum width ceiling for wide documentation/manuals.
`padding`	`ImVec2`	`(8.0f, 6.0f)`	Internal window padding around the tooltip contents.
`item_spacing`	`ImVec2`	`(4.0f, 0.0f)`	Item spacing between lines. `y = 0.0f` matches native font leading.
`frame_padding`	`ImVec2`	`(2.0f, 1.0f)`	Internal padding for frame elements inside the tooltip.
`indent_spacing`	`float`	`20.0f`	Standard indent spacing between nested list levels.
`bullet_spacing`	`float`	`4.0f`	**[Markdown Tooltips Only]** Gap in pixels between bullet glyph and following text when rendering rich lists via `TextMD`. (Ignored for plain-text tooltips.)
`font_scale`	`float`	`0.0f`	Window font scale multiplier (`0.0f` = automatically inherit the ambient font scale from the calling parent window; `>0.0f` = explicit scale override).
`bg_color`	`ImVec4`	Solid yellow	Window and popup background color (`#FFF299`, 100% opaque `alpha = 1.0f`).
`border_color`	`ImVec4`	Dark yellow	Tooltip border outline color (`#CCBF66`).
`text_color`	`ImVec4`	Jet black	Tooltip text and separator color (`#000000`).

NOTE

Resolution-Independent Base Dimensions: All dimensional settings (`default_wrap_width`, `min_wrap_width`, `max_wrap_width`, `padding`, `item_spacing`, `frame_padding`, and `indent_spacing`) are defined in **base design boxels** (at 1.0× baseline). The tooltip engine resolves the active ambient scale (`ImGui::GetFontSize() / 14.0f` or explicit `font_scale`) and automatically scales constraints and padding dynamically. Callers do not need to manually multiply configuration values by font zoom factors.

Usage Patterns: Global Setup vs. Per-Call Overrides

Pattern A: Global Setup (Once at Plugin Startup)

For 95% of use cases, configure your desired tooltip personality once during plugin startup or window creation:

```cpp
// In your plugin initialization:
static ImGui::TimedTooltip::Config s_MyTooltipConfig;
s_MyTooltipConfig.hover_delay_cycles = 45;                          // Snappier hover delay
s_MyTooltipConfig.padding            = ImVec2(8.0f, 6.0f);          // Compact padding
s_MyTooltipConfig.item_spacing       = ImVec2(4.0f, 0.0f);          // Tight line pitch
s_MyTooltipConfig.bg_color           = ImVec4(1.0f, 0.95f, 0.6f, 1.0f); // 100% opaque sticky note

ImGui::TimedTooltip::SetDefaultTooltipConfig(&amp;s_MyTooltipConfig);
```

Every subsequent call to **`ImGui::TimedTooltip::Text(...)`** and **`ImGui::TimedTooltip::TextMD(...)`** will automatically use this configuration without needing any per-call arguments.

Pattern B: Per-Call Override (Instant & Dynamic)

If a specific widget needs unique styling—for example, a wider maximum width, distinct background color, or custom padding—you can fetch a copy of the active configuration using **`ImGui::TimedTooltip::GetConfig()`**, tweak the desired fields, and pass it directly to **`Text`** or **`TextMD`**:

```cpp
// Safely copies the active global config (or built-in defaults if no global config was set):
ImGui::TimedTooltip::Config wideConfig = ImGui::TimedTooltip::GetConfig();
wideConfig.max_wrap_width = 800.0f;                       // Allow wider manual formatting
wideConfig.bg_color       = ImVec4(0.15f, 0.15f, 0.20f, 0.95f); // Dark HUD styling
wideConfig.text_color     = ImVec4(1.0f, 1.0f, 1.0f, 1.0f);

ImGui::Button("System Diagnostics");
// Pass custom configuration directly:
ImGui::TimedTooltip::TextMD(&amp;wideConfig, "### System Status\n* Bus 1: **Normal**\n* Bus 2: **Normal**")
```

Pattern C: Low-Level Custom Layout Blocks

For completely custom tooltip content containing arbitrary ImGui widgets, use **`BeginStationaryTooltip`** and **`EndStationaryTooltip`**:

```cpp
ImGui::Button("Engine Monitor");
if (ImGui::TimedTooltip::BeginStationaryTooltip("EngineMonitorTooltip")) {
    ImGui::Text("EGT: 640 C");
    ImGui::ProgressBar(0.75f, ImVec2(150, 15));
    ImGui::TimedTooltip::EndStationaryTooltip("EngineMonitorTooltip");
}
```

Hermetic Style Isolation

In Dear ImGui, child windows and popups can inadvertently inherit active layout styles from their parent windows. If an outer menu bar or custom toolbar has set **`ItemSpacing.x = 0.0f`** or compressed **`FramePadding`** to pack buttons tightly, standard tooltips would inherit those distorted spacing values.

`TimedTooltip` isolates itself completely:

1. When opening a tooltip proxy, **`BeginStationaryTooltipProxy`** pushes:

- **`ImGuiStyleVar_WindowPadding`** (**`config->padding`**)

- `ImGuiStyleVar_ItemSpacing` (`config->item_spacing`)
- `ImGuiStyleVar_FramePadding` (`config->frame_padding`)
- `ImGuiStyleVar_IndentSpacing` (`config->indent_spacing`)

2. When closing, `EndStationaryTooltip` pops all four style variables in exact reverse order.

This guarantees that:

- **No outer window hacks leak inward:** Tooltips always render with crisp, predictable typography regardless of what the calling window modified.
- **No tooltip styles leak outward:** Calling windows retain their exact style state without needing defensive push/pop blocks.

Screen Clamping & Docking

Tooltips automatically check the active display dimensions (`ImGui::GetIO().DisplaySize`):

- **Bottom Edge Protection:** If `render_pos.y + height` exceeds the screen floor, the anchor position automatically shifts upward so the entire tooltip remains visible on screen.
- **Right Edge Protection:** If `render_pos.x + width` exceeds the screen edge, the tooltip shifts leftwards to keep all content in view.

Click Forwarding & Automatic Dismissal

Tooltips attached to interactive or proxy-instrumented widgets (like `ButtonLink`, chevron menus, or `TextProxy` / `TextMDProxy` items) automatically intercept clicks via `ImGui::IsItemClicked()`:

- **Immediate Dismissal:** Clicking an active widget immediately dismisses the active tooltip, preventing tooltips from lingering over open menus or blocking post-click interactions.
- **Toggle on Idle Click:** Clicking an idle/non-actionable widget (such as a dimmed reset button or standalone info link) cleanly toggles the explanation tooltip on/off.

X-Plane 12 Panel Graphics (Linear Blending & Font Scaling)

1. 100% Solid Opacity (`alpha = 1.0f`)

In legacy OpenGL, plugins composited in uncorrected gamma space, where semi-transparent yellow cards (`alpha = 0.95f`) appeared largely solid.

In X-Plane 12's modern **Panel Graphics** pipeline (Metal on macOS, Vulkan on Windows/Linux), compositing is computed in **physically linear color space**. Even a minor 5% transparency allows intense runway lights, aircraft labels, and bright background elements to illuminate the pixels under dark text. Furthermore, linear antialiasing blends 50% edge coverage at $\approx 73\%$ background luminance ($(0.5)^{1/2.2} \approx 0.73$), optically eroding dark letter stems.

To ensure crisp, dark typography without background wash-out, `TooltipConfig::bg_color` defaults to `1.0f` **solid opacity** (`ImVec4(1.0f, 0.95f, 0.6f, 1.0f)`).

2. Automatic Ambient Font Scale Inheritance

When `TooltipConfig::font_scale` is left at `0.0f` (the default), the tooltip engine automatically queries and inherits the ambient window font scale of the calling widget:

- **Custom Window Zoom:** If a child window or floating palette is zoomed to 124%, hovering a widget inside that window renders the tooltip scaled at that exact same 124% factor.

- **Accessibility / Senior Citizen Mode:** If a global or per-window 17pt font mode is active, the tooltip inherits the enlarged scale seamlessly.
- **Pre-Calculation Accuracy:** The 2-pass size pre-calculator (**`MD::CalcMarkdownSize`**) computes wrap boundaries and vertical heights using this exact inherited scale factor, guaranteeing zero text clipping and zero scrollbar flutter.

API Summary

```
[CPP]
namespace ImGui {
    namespace TimedTooltip {
        // High-level Plain Text (auto-wraps to TooltipConfig bounds; returns true when displayed):
        bool Text(const char* fmt, ...);
        bool Text(const TooltipConfig* config, const char* fmt, ...);
        bool Text(const std::string&amp; text, const TooltipConfig* config = nullptr);

        // Single-line / Non-Wrapping Plain Text (mirrors ImGui::TextUnformatted):
        bool TextUnformatted(const char* text, const char* text_end = nullptr, const TooltipConfig* config
        bool TextUnformatted(const std::string&amp; text, const TooltipConfig* config = nullptr);

        // High-level Plain Text Proxy (explicit ID and hover state):
        bool TextProxy(ImGuiID id, bool is_hovered, const char* fmt, ...);
        bool TextProxy(ImGuiID id, bool is_hovered, const TooltipConfig* config, const char* fmt, ...);
        bool TextProxy(ImGuiID id, bool is_hovered, const std::string&amp; text, const TooltipConfig* confi
        bool TextProxy(const char* str_id, bool is_hovered, const char* fmt, ...);
        bool TextProxy(const char* str_id, bool is_hovered, const std::string&amp; text, const TooltipConfi
        bool TextUnformattedProxy(ImGuiID id, bool is_hovered, const char* text, const char* text_end = nul

        // High-level Markdown (requires imgui_markdown_ext.h in either include order; returns true when di

        bool TextMD(const char* fmt, ...);
        bool TextMD(const TooltipConfig* config, const char* fmt, ...);
        bool TextMD(const std::string&amp; markdown_text, const TooltipConfig* config = nullptr);

        // Single-line / Non-Wrapping Markdown (never wraps unless \n or PushTextWrapPos is active):
        bool TextUnformattedMD(const char* text, const char* text_end = nullptr, const TooltipConfig* confi
        bool TextUnformattedMD(const std::string&amp; markdown_text, const TooltipConfig* config = nullptr)

        // High-level Markdown Proxy (explicit ID and hover state):
        bool TextMDProxy(ImGuiID id, bool is_hovered, const char* fmt, ...);
        bool TextMDProxy(ImGuiID id, bool is_hovered, const TooltipConfig* config, const char* fmt, ...);
        bool TextMDProxy(const char* str_id, bool is_hovered, const char* fmt, ...);
        bool TextMDProxy(ImGuiID id, bool is_hovered, const std::string&amp; markdown_text, const TooltipCo
        bool TextMDProxy(const char* str_id, bool is_hovered, const std::string&amp; markdown_text, const T
        bool TextUnformattedMDProxy(ImGuiID id, bool is_hovered, const char* text, const char* text_end = n

        // Explicitly opt the preceding widget into click-to-toggle tooltip behavior for the current frame:
        void ToggleOnClick(bool enable = true);

        // Configuration:

        using Config = TooltipConfig;
        using State  = TooltipState;
        void SetDefaultTooltipConfig(const Config* config);
        const Config*&amp; GetDefaultTooltipConfig();
        const Config&amp; GetCurrentTooltipConfig();
        Config GetConfig();

        // Low-level lifecycle controls:
        bool BeginStationaryTooltipProxy(ImGuiID id, bool is_hovered, const Config* cfg = nullptr);
        bool BeginStationaryTooltip(ImGuiID id, const Config* cfg = nullptr);
        bool BeginStationaryTooltip(const char* str_id, const Config* cfg = nullptr);
        void EndStationaryTooltip(ImGuiID id);
        void EndStationaryTooltip(const char* str_id);
        void ClearActiveTooltip();
    }
}
```